

LAB1: Intro & Getting Started

Efficient Computing for Artificial Intelligence

Valentino Peluso
valentino.peluso@polito.it

What you will learn



- Develop ML pipelines for IoT applications
 - Collect data with sensors
 - Store & process data
 - Train & optimize ML models for IoT data
 - Deploy ML models at the IoT edge
 - Implement edge-to-cloud/cloud-to-edge communication
- Programming language:
 - Python
- Tools/IDEs:
 - Visual Studio Code (local development)
 - Deepnote (cloud development)
 - Redis (cloud storage)

LAB Schedule

Part I

DSE & CE

- **LAB1:** Getting started
 - 9 Oct, 16:00-19:00 – Room 5I
- **LAB2:** Distributed Computing
 - 16 Oct, 16:00-19:00 – Room 5I
- **Homework #1:**
 - 23 Oct, 16:00-19:00 – Room 5I
 - 30 Oct, 16:00-19:00 – Room 5I
 - **Submission deadline: 26 Nov**

Part II

DSE & CE

- **LAB3:** Training & Deployment
 - 6 Nov, 16:00-19:00 – Room 5I
 - 13 Nov, 16:00-19:00 – Room 5I
- **LAB4:** Optimization
 - 20 Nov, 16:00-19:00 – Room 5I
- **Homework #2**
 - 27 Nov, 16:00-19:00 – Room 5I
 - 4 Dec, 16:00-19:00 – Room 5I
 - **Submission deadline: 7 Jan**

Part III

DSE

- **LAB5:** Communication
 - 11 Dec, 16:00-19:00 – Room 5I
- **Homework #3**
 - 18 Dec, 16:00-19:00 – Room 5I
 - 8 Jan, 16:00-19:00 – Room 5I
 - **Submission deadline: 31 Jan**

LAB Evaluation

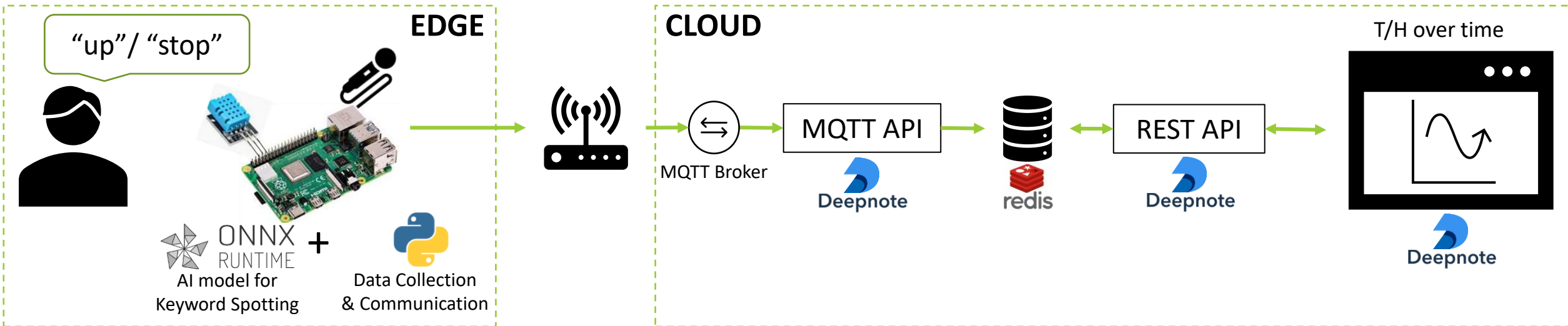
- Homework submissions are **mandatory**
- Deliverables:
 - Code (Python scripts/notebooks)
 - 1-page PDF report
- Evaluation:
 - DSE:
 - 6 points (max.) for each homework (x3)
 - CE:
 - 9 points (max.) for each homework (x2)
 - **18** (over 32) points total

Use Case: Smart Hygrometer

- Remote monitoring of Temperature/Humidity (T/H)
- The User can start/stop the monitoring using voice commands
- The User can visualize the T/H over time from a Web Application



System Architecture

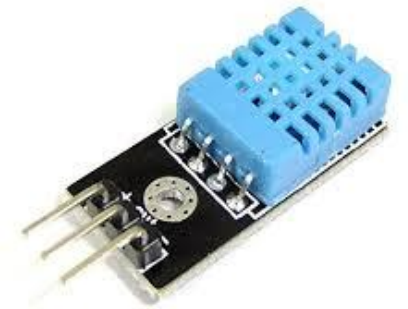


Local Environment: RPI Kit

- Sensing:
 - USB microphone
 - DHT-11 Temperature & Humidity sensor
- Processing: Raspberry Pi 4 Model B 2GB
 - Visual Studio Code
 - Python 3.11



USB Mic.



DHT-11



Raspberry Pi



SD Card



Ethernet Cable



RPI Power Supply

Cloud Environment

- Processing: Deepnote
 - Collaborative Jupyter-like platform for Data Science teams
- Storage: Redis Cloud
 - Key-Value Database
- Communication: Eclipse MQTT Broker
 - Message broker for MQTT protocol

Preparation Status

1. Create a group (deadline **today** - Oct. 9th): [link](#)
 2. Sign up for Deepnote: accept the invitation received in your student inbox
 3. Sign up for Redis Cloud: <https://redis.io/try-free>
 4. Download VSCode: <https://code.visualstudio.com/download>
- Please reply to the poll at this [link](#)
 - Next Steps:
 - Follow the instructions in **LAB1-Exercises.pdf**
(Material → Labs → lab1-getting-started)



Group List

- The final group list has been uploaded to the *Material* page of the course
 - File: ECAI 26 Group List 9.10.pdf
- Check your group's member
- For any changes, please send a message to valentino.peluso@polito.it

Group	Surname Name	Student ID
1	Toprak Mert	s336966
	Leffel Daghan Ufuk	s338142
	Oral Muhammed Emin	s343124
2	Papalia Pasquale	s344701
	Rosapepe Emanuele	s346705
	Termino Nicolò	s338680
3	Basaran Samet	s340721
	Celik Murat	s338212
	Durante Lorenzo	s347143
4	Deng Lan	s338219
	Mastromauro Fabio	s343750
	Kevser Gunaydin	s338221
5	Calabrese Lorenzo	s345902
	Caffapietro Flavia	s345862
	Spedicato Giuseppe	s337116
6	Scolaro Erik	s343553
	Martino Gabriele	s343522
	De Gregorio Lorenzo	s339344
7	Di Felice Andrea	s337517
	Tedesco Valeria	s340892
	Ciorra Edoardo	s337049
8	Ascheri Simone	s347257
	Licalzi Nunzio	s344860
	Lolli Andrea	s346165
9	Lucia Asia Alessia	s349149
	Vercellone Romeo	s341967
	Perlo Alessandro	s337131
10	Sara Beatrice	s316683
	Giuseppe Ventrella	s348517
	Emanuele Romito	s339170
11	Arianna Coppola	s317870
	Hatem Mohamed	s346906
	Luciano Scarpino (Data Science)	s346205
12	Alessio Berardi	s343182

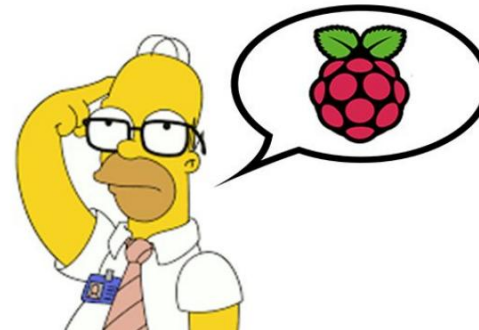
Time to grab your Raspberry Pi 🎉

One student per group, please come forward with your **Student Card** to collect your **Raspberry Pi kit**.

Expectation:



Reality:



Please take good care of your kit
It must be returned at the end of the semester!

Cloud Environment Setup: Deepnote

The image shows a screenshot of the Deepnote website and a preview of its notebook interface. The website header includes navigation links for Pricing, Enterprise, Book a demo, and a Get started button. A green callout box points to the URL 'deepnote.com'. The main content area features a large heading 'Introducing notebooks: The AI-powered data workspace' and a list of features: Deepnote Agent with GPT-5, Claude Sonnet 4, and your data context; turning notebooks into data apps or dashboards; combining Python, SQL, R, or no code; and querying Snowflake, BigQuery, CSV, and 50 more data sources. A 'Get started - it's free' button is prominent. An inset shows a notebook titled 'Edison Insights' with a 'Telco data exploration' tab. It displays a 'Ready' status, a table of 100 rows, and a code editor with Python code for data analysis. A 'Watch demo (5 min)' button is also visible. The bottom of the notebook interface shows tabs for 'Any data sources', 'AI suggestions', and 'Interactive dashboards'.

Deepnote: Analytics and d

deepnote.com

Go to deepnote.com

NEW Deepnote Agent is now available! →

Introducing notebooks: The AI-powered data workspace

- Deepnote Agent with **GPT-5**, **Claude Sonnet 4**, and your data context
- Turn **notebooks** into powerful **data apps** or **dashboards**
- Combine **Python**, **SQL**, **R**, or don't write code at all
- Query **Snowflake**, **BigQuery**, **CSV**, and **50 more data sources**

Get started – it's free

Book a demo

Edison Insights

Telco data exploration

Ready

100 rows, showing 10 per page

Explore how monthly charges and total charges relate to other factors like contract type, tech support, and internet service.

```
# Let's start by exploring how monthly charges relate to contract type, tech support, and internet service using boxplots
import matplotlib.pyplot as plt
import seaborn as sns

fig, axes = plt.subplots(1, 2)
sns.boxplot(x='contract_type', y='monthly_charges', data=telco_data)
sns.boxplot(x='contract_type', y='total_charges', data=telco_data)
```

Watch demo (5 min)

ENVIRONMENT

Python 3.9

RAM: 0.5 GB

CPU: 0%

SQL

Any data sources

AI suggestions

Interactive dashboards

Cloud Environment Setup: Deepnote

The image shows a screenshot of the Deepnote website and a preview of its notebook interface. The website header includes navigation links for Platform, Customers, Resources, Docs, and a 'Get started' button. A green callout box points to this button with the text 'Click "Get Started"'. The main content area features the headline 'Introducing notebooks: The AI-powered data workspace' and a list of features: Deepnote Agent with GPT-5, Claude Sonnet 4, and your data context; turning notebooks into data apps or dashboards; combining Python, SQL, R, or no code; and querying Snowflake, BigQuery, CSV, and 50 more data sources. A 'Get started - it's free' button is at the bottom. An inset shows a notebook titled 'Edison Insights' with a sidebar for notebooks, integrations, files, terminals, and variable explorer. The main area contains a code editor with Python code for data exploration and a 'Watch demo (5 min)' button. The bottom of the inset shows tabs for 'Any data sources', 'AI suggestions', and 'Interactive dashboards'.

Deepnote: Analytics and d

deepnote.com

Platform Customers Resources Docs

Deepnote

Pricing Enterprise Book a demo **Get started**

NEW Deepnote Agent is now available! →

Introducing notebooks: The AI-powered data workspace

- Deepnote Agent with **GPT-5**, **Claude Sonnet 4**, and your data context
- Turn **notebooks** into powerful **data apps** or **dashboards**
- Combine **Python**, **SQL**, **R**, or don't write code at all
- Query **Snowflake**, **BigQuery**, **CSV**, and **50 more data sources**

Get started – it's free Book a demo

Edison Insights

Telco data exploration

NOTEBOOKS +
Notebook 1

INTEGRATIONS +
Snowflake demo

FILES > +

TERMINALS +

VARIABLE EXPLORER > 2

ENVIRONMENT
Python 3.9
RAM: 0.5 GB CPU: 0%
SQL

Ready

100 rows, showing 10 per page

Explore how monthly charges and total charges relate to other factors like contract type, tech support, and internet service.

```
# Let's start by exploring how monthly charges relate to contract type, tech support, and internet service using br
import matplotlib.pyplot as plt
import seaborn as sns

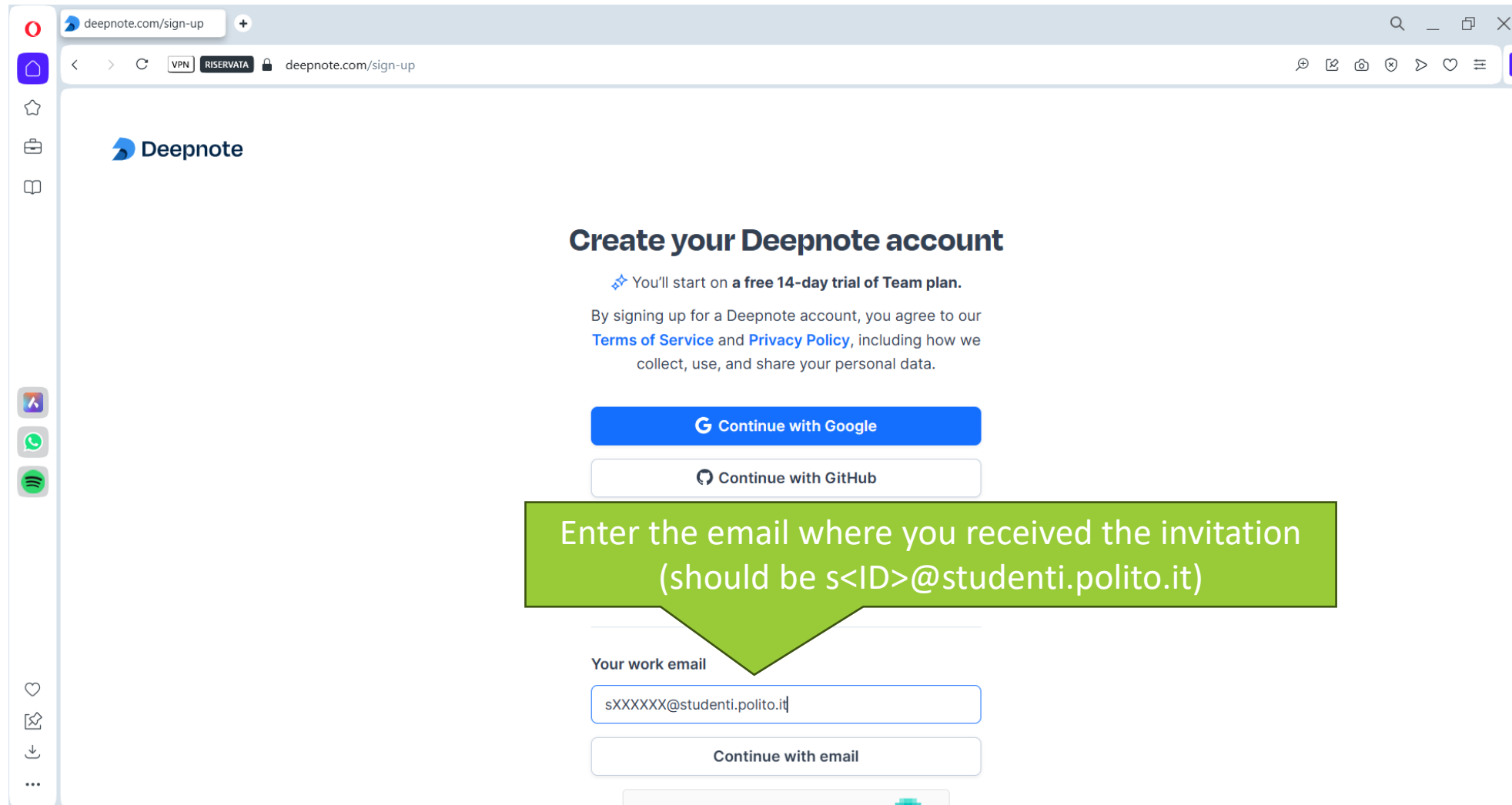
fig, axes = plt.su
sns.boxplot(x='con
```

Watch demo (5 min)

AI Code Text SQL Chart Input

Any data sources AI suggestions Interactive dashboards

Cloud Environment Setup: Deepnote



The screenshot shows the Deepnote sign-up page in a web browser. The page title is "Create your Deepnote account". Below the title, it says "You'll start on a free 14-day trial of Team plan." and "By signing up for a Deepnote account, you agree to our Terms of Service and Privacy Policy, including how we collect, use, and share your personal data." There are two buttons: "Continue with Google" (blue) and "Continue with GitHub" (white). A green callout box with a downward-pointing arrow contains the text: "Enter the email where you received the invitation (should be s<ID>@studenti.polito.it)". Below the callout, there is a label "Your work email" and a text input field containing "sXXXXXX@studenti.polito.it". At the bottom, there is a button "Continue with email".

deepnote.com/sign-up

Deepnote

Create your Deepnote account

🌟 You'll start on a free 14-day trial of Team plan.

By signing up for a Deepnote account, you agree to our [Terms of Service](#) and [Privacy Policy](#), including how we collect, use, and share your personal data.

[Continue with Google](#)

[Continue with GitHub](#)

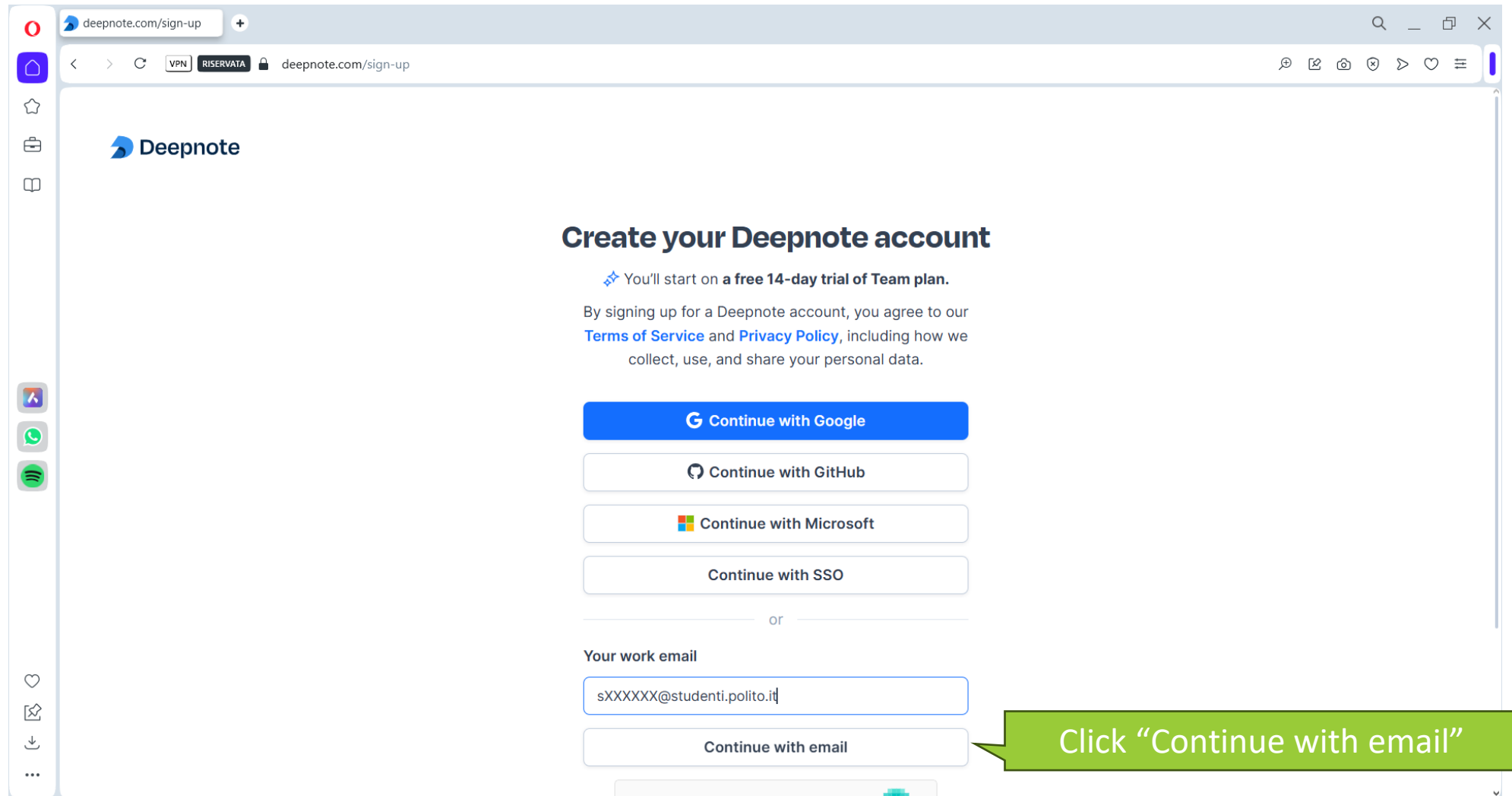
Enter the email where you received the invitation
(should be s<ID>@studenti.polito.it)

Your work email

sXXXXXX@studenti.polito.it

[Continue with email](#)

Cloud Environment Setup: Deepnote



The screenshot shows a web browser window with the URL `deepnote.com/sign-up`. The page features the Deepnote logo and a heading "Create your Deepnote account". Below the heading, it states: "You'll start on a free 14-day trial of Team plan." and "By signing up for a Deepnote account, you agree to our [Terms of Service](#) and [Privacy Policy](#), including how we collect, use, and share your personal data." There are four buttons for social login: "Continue with Google", "Continue with GitHub", "Continue with Microsoft", and "Continue with SSO". Below these is a section for email sign-up with the label "Your work email", a text input field containing "sXXXXXX@studenti.polito.it", and a "Continue with email" button. A green callout bubble points to the "Continue with email" button with the text "Click 'Continue with email'".

deepnote.com/sign-up

Deepnote

Create your Deepnote account

🌟 You'll start on a **free 14-day trial of Team plan**.

By signing up for a Deepnote account, you agree to our [Terms of Service](#) and [Privacy Policy](#), including how we collect, use, and share your personal data.

[Continue with Google](#)

[Continue with GitHub](#)

[Continue with Microsoft](#)

[Continue with SSO](#)

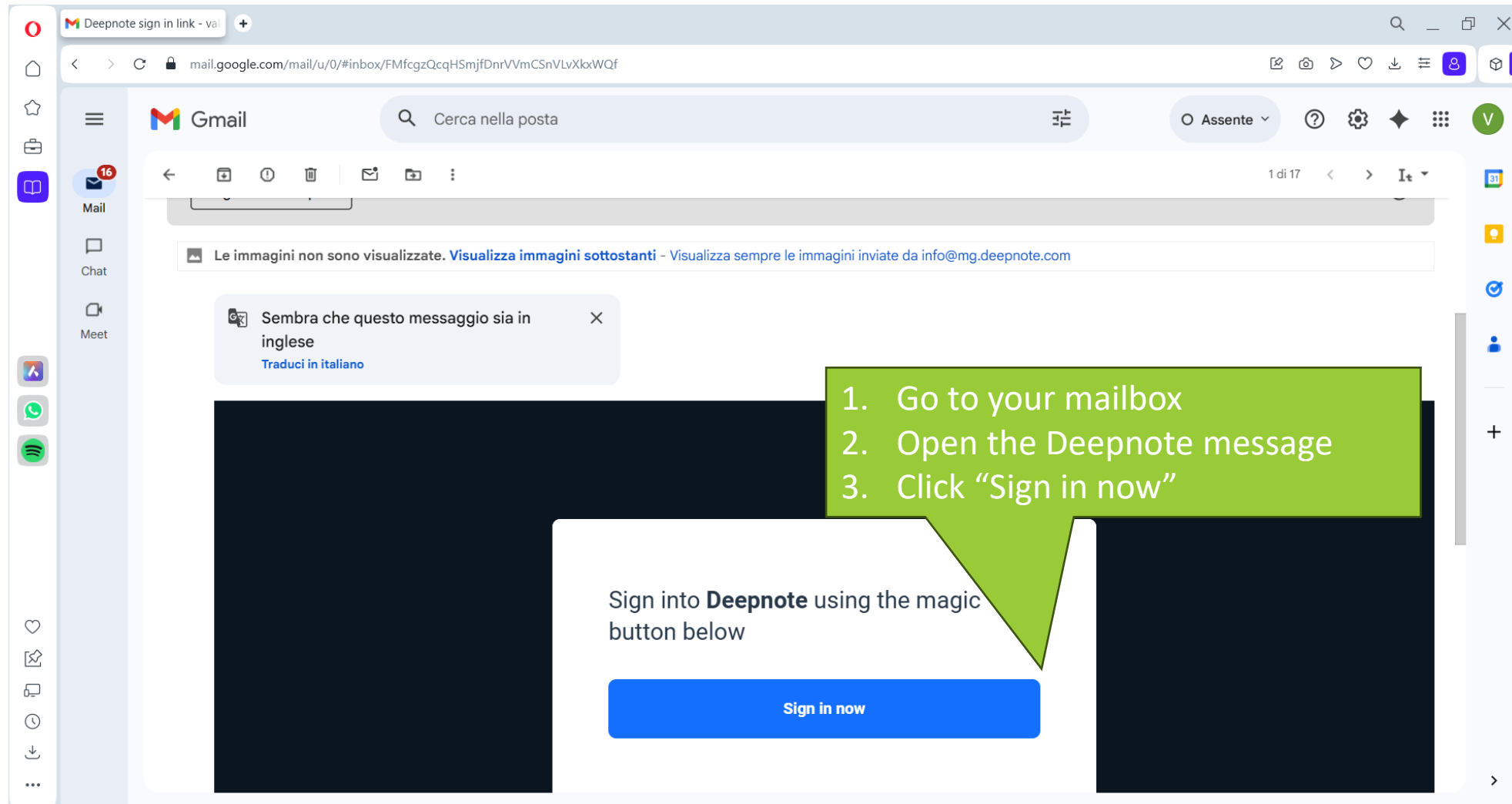
or

Your work email

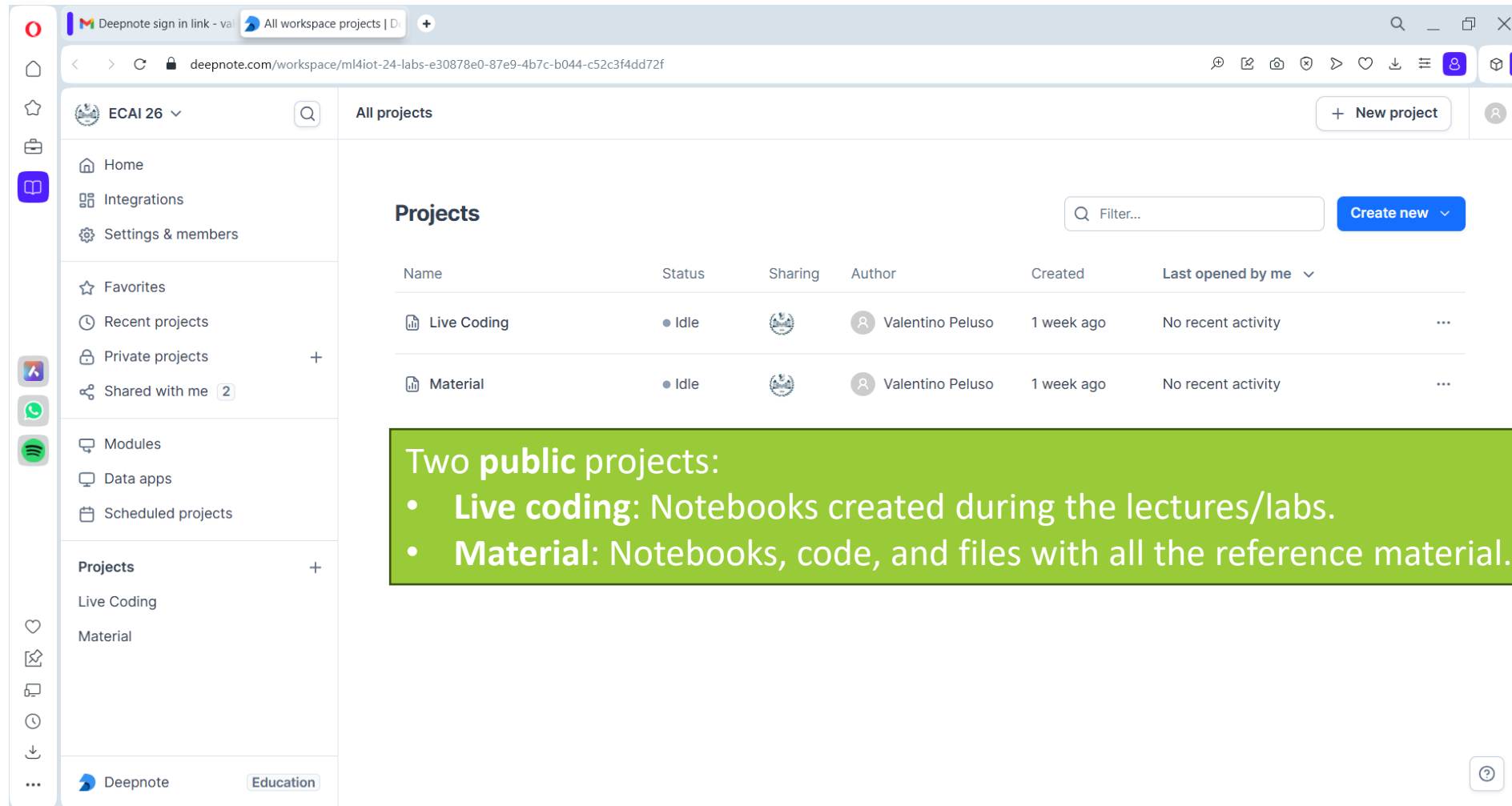
[Continue with email](#)

Click "Continue with email"

Cloud Environment Setup: Deepnote



Cloud Environment Setup: Deepnote



The screenshot shows the Deepnote web interface. The left sidebar contains navigation links: Home, Integrations, Settings & members, Favorites, Recent projects, Private projects, Shared with me (2), Modules, Data apps, Scheduled projects, Projects (with a sub-list of Live Coding and Material), and Deepnote Education. The main area displays a table of projects under the heading 'All projects'. The table has columns for Name, Status, Sharing, Author, Created, and Last opened by me. Two projects are listed: 'Live Coding' and 'Material', both with a status of 'Idle' and created '1 week ago' by 'Valentino Peluso'. A green callout box highlights these two projects.

Projects

Name	Status	Sharing	Author	Created	Last opened by me
Live Coding	Idle		Valentino Peluso	1 week ago	No recent activity
Material	Idle		Valentino Peluso	1 week ago	No recent activity

Two **public** projects:

- **Live coding:** Notebooks created during the lectures/labs.
- **Material:** Notebooks, code, and files with all the reference material.

Deepnote: Create a Private Project

The screenshot shows the Deepnote workspace interface. The left sidebar contains navigation options: Home, Integrations, Settings & members, Favorites, Recent projects, Private projects (highlighted with a green callout), Shared with me (2), Modules, Data apps, and Scheduled projects. The main area displays a table of projects under the heading 'Projects'. The table has columns for Name, Status, Sharing, Author, Created, and Last opened by me. The first project listed is 'Live Coding' by Valentino Peluso, created 1 week ago, with a status of 'Idle'.

Click "+" next to "Private projects"

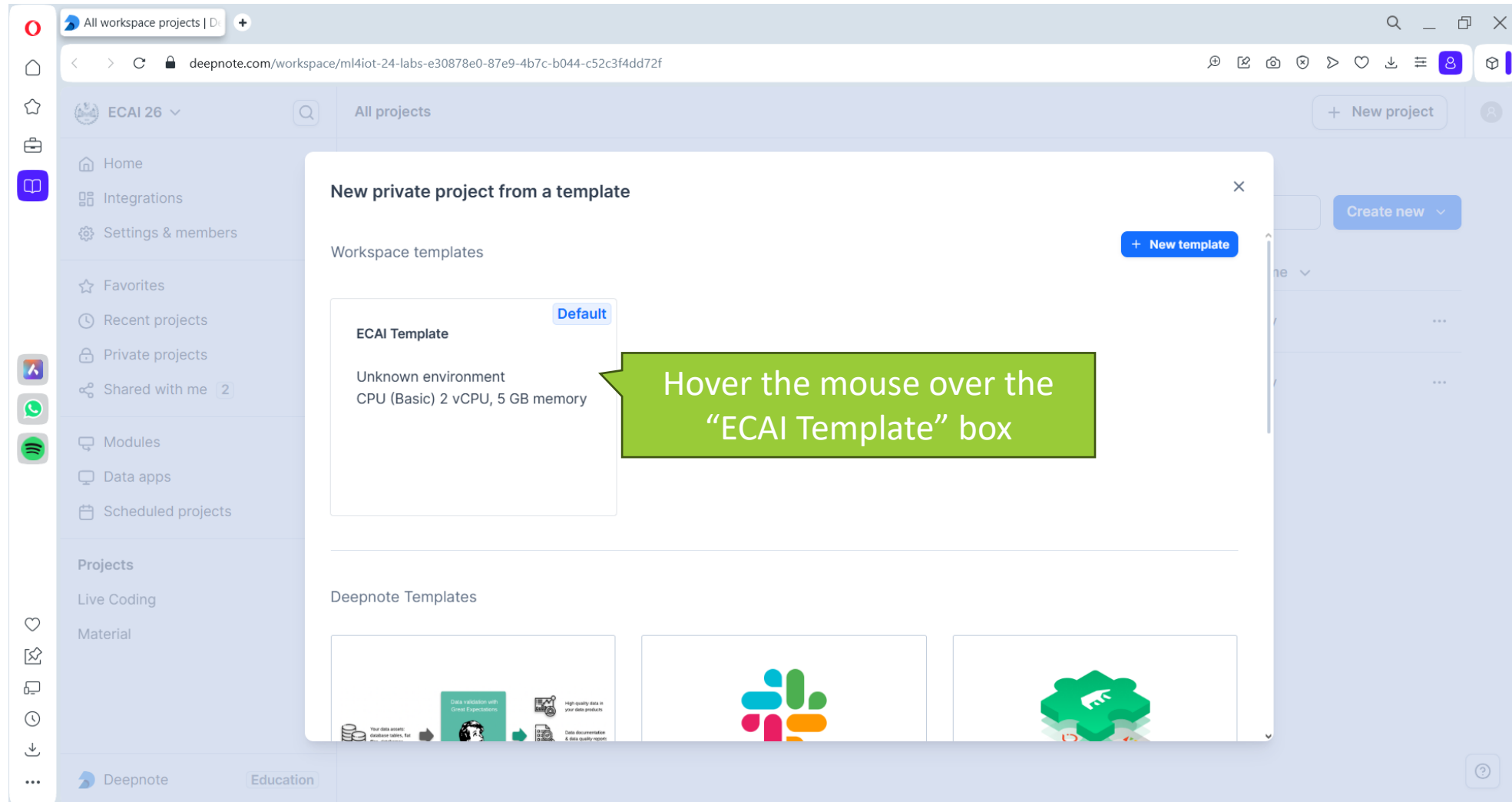
Name	Status	Sharing	Author	Created	Last opened by me
Live Coding	Idle		Valentino Peluso	1 week ago	No recent activity

Deepnote: Create a Private Project

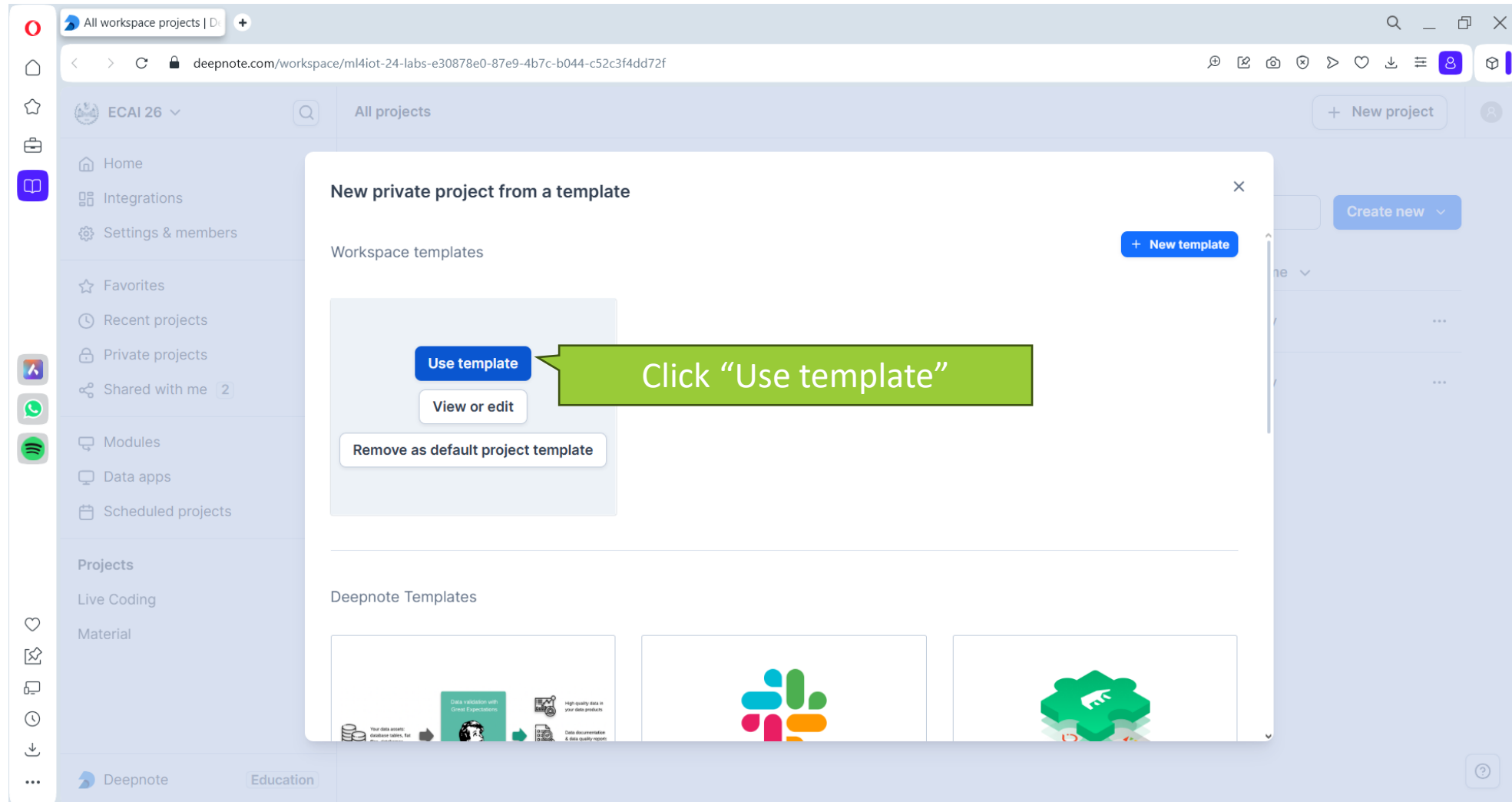
The screenshot shows the Deepnote workspace interface. The left sidebar contains navigation links: Home, Integrations, Settings & members, Favorites, Recent projects, Private projects (highlighted with a plus icon), Shared with me (2), Modules, Data apps, and Scheduled projects. The main area displays 'All projects' with a table of existing projects. A dropdown menu is open over the 'Private projects' link, showing options: 'New project', 'New empty project', and 'New project from template'. A green callout box with the text 'Click "New project from template"' points to this option.

Name	Status	Sharing	Author	Created	Last opened by me
Live Coding	Idle		Valentino Peluso	1 week ago	No recent activity
	Idle		Valentino Peluso	1 week ago	No recent activity

Deepnote: Create a Private Project



Deepnote: Create a Private Project



Deepnote: Create a Private Project

The screenshot displays the Deepnote web interface. The browser address bar shows the URL: `deepnote.com/workspace/ECAI-26-e30878e0-87e9-4b7c-b044-c52c3f4dd72f/project/Valentino-Peluso-Tests-Untitled-project-94727b9a-1607-4a9c-a3d1-798fd357db4d/notebc`. The project name "My Private Project" is highlighted in the top navigation bar, with a green callout box pointing to it that says "Enter a project name".

The left sidebar contains several sections:

- Notebooks:** Includes "Init" and "Notebook 1".
- Integrations:** A section to "Connect an integration" with a subtext: "Select from existing ones, or add a new one. View its schema or query it with SQL." It shows icons for various services like AWS, Google, and Spotify.
- Files:** Includes a "Dockerfile" icon.
- Machine:** Shows the current machine configuration: "CPU (Basic)" with "2 vCPU, 5 GB memory". It also has a "Dockerfile" dropdown and a "Stop machine" button.

The main workspace area shows a code editor with the following Python code:

```
1 import sys
2 import torch
3
4 print(sys.version)
5 if sys.version.split()[0] != '3.12.7':
6     raise RuntimeError('Wrong Python version. Expected Version is 3.12.7.')
7
8 print('Python version: OK\n')
9
10 print(torch.__version__)
11 if torch.__version__ != '2.7.0+cu126':
12     raise RuntimeError('Wrong PyTorch version. Expected Version is 2.7.0+cu126.')
13
14 print('PyTorch version: OK')
```

Below the code editor, the terminal output shows the execution results:

```
3.12.7 (main, Jul 18 2025, 11:18:59) [GCC 10.2.1 20210110]
Python version: OK

2.7.0+cu126
PyTorch version: OK
```

At the bottom of the interface, there are tabs for "Code", "Text", "SQL", "Chart", "Input", and "Module". The "Code" tab is currently selected.

Deepnote: Create a Private Project

The screenshot displays the Deepnote web interface for a private project. The left sidebar contains navigation options: Notebooks (with 'Init' and 'Notebook 1'), Integrations (with a 'Connect an integration' button), Files (with a 'Dockerfile' button), and Machine configuration (showing 'CPU (Basic)' with '2 vCPU, 5 GB memory' and a 'Stop machine' button). The main workspace is titled 'My Private Project' and features a 'Congratulations, you ran your first notebook' notification. A 'Run' button is visible above the code editor. The code editor contains Python code that checks the system's Python and PyTorch versions. A green callout box points to the 'Run' button with the text 'Click "Run" to execute the notebook'. The output area shows the execution results, including the Python and PyTorch versions and the message 'PyTorch version: OK'.

My Private Project | Deepnote

deepnote.com/workspace/ECAI-26-e30878e0-87e9-4b7c-b044-c52c3f4dd72f/project/Valentino-Peluso-Tests-Untitled-project-94727b9a-1607-4a9c-a3d1-798fd357db4d/notebook

ECAI 26

Notebooks

Init

Notebook 1

Integrations

Connect an integration

Select from existing ones, or add a new one. View its schema or query it with SQL.

Files

Dockerfile

Machine

3s 0% 0GB

CPU (Basic)

2 vCPU, 5 GB memory

Dockerfile

Stop machine

My Private Project

Congratulations, you ran your first notebook 🎉!

Share

Starting up the machine

Run

Create app

Click "Run" to execute the notebook

```
1 import sys
2 import torch
3
4 print(sys.version)
5 if sys.version.split()[0] != '3.12.7':
6     raise RuntimeError('Wrong Python version. Expected Version is 3.12.7.')
7
8 print('Python version: OK\n')
9
10 print(torch.__version__)
11 if torch.__version__ != '2.7.0+cu126':
12     raise RuntimeError('Wrong PyTorch version. Expected Version is 2.7.0+cu126.')
13
14 print('PyTorch version: OK')
```

In queue

3.12.7 (main, Jul 18 2025, 11:18:59) [GCC 10.2.1 20210110]

Python version: OK

2.7.0+cu126

PyTorch version: OK

Code

Text

SQL

Chart

Input

Module

New

Deepnote: Create a Private Project

The screenshot shows the Deepnote web interface for a private project. The left sidebar includes sections for Notebooks (Init, Notebook 1), Integrations (Connect an integration), Files (Dockerfile), and Machine (CPU Basic, 2 vCPU, 5 GB memory, Dockerfile, Stop machine). The main workspace shows a Python script in 'Notebook 1' that checks the system and PyTorch versions. The script is as follows:

```
1 import sys
2 import torch
3
4 print(sys.version)
5 if sys.version.split()[0] != '3.12.7':
6     raise RuntimeError('Wrong Python version. Expected Version is 3.12.7.')
7
8 print('Python version: OK\n')
9
10 print(torch.__version__)
11 if torch.__version__ != '2.7.0+cu126':
12     raise RuntimeError('Wrong PyTorch version. Expected Version is 2.7.0+cu126.')
13
14 print('PyTorch version: OK')
```

The output of the script is displayed below the code:

```
3.12.7 (main, Sep 11 2025, 12:40:23) [GCC 10.2.1 20210110]
Python version: OK

2.7.0+cu126
PyTorch version: OK
```

A green callout box with the text "Check the expected output to validate your setup" points to the output of the script.

Deepnote: Create a Private Project

The screenshot displays the Deepnote web interface for a private project named "My Private Project". The interface includes a sidebar with navigation options like "Notebooks", "Integrations", "Files", and "Machine". The main area shows a code editor with Python code for checking system versions. A "Share" button is visible in the top right. A modal window is open for sharing the project, showing a list of members and an "Invite" button. Three green callout boxes provide instructions: 1. Click "Share" to share the project with your groupmates. 2. Enter their email addresses or names. 3. Click "Invite".

1. Click "Share" to share the project with your groupmates

2. Enter their email addresses or names

3. Click "Invite"

```
1 import sys
2 import torch
3
4 print(sys.version)
5 if sys.version.split()[0] != '3.12.7':
6     raise RuntimeError('Wrong Python version. Expected 3.12.7')
7
8 print('Python version: OK\n')
9
10 print(torch.__version__)
11 if torch.__version__ != '2.7.0+cu126':
12     raise RuntimeError('Wrong PyTorch version. Expected 2.7.0+cu126')
13
14 print('PyTorch version: OK')
```

3.12.7 (main, Sep 11 2025, 12:40:23) [GCC 10.2.1 20210110]
Python version: OK

2.7.0+cu126
PyTorch version: OK

Members at ECAI 26
Project is private, workspace members can't access it

Valentino Peluso Test
s206803@studenti.polito.it
Admin (Project owner)

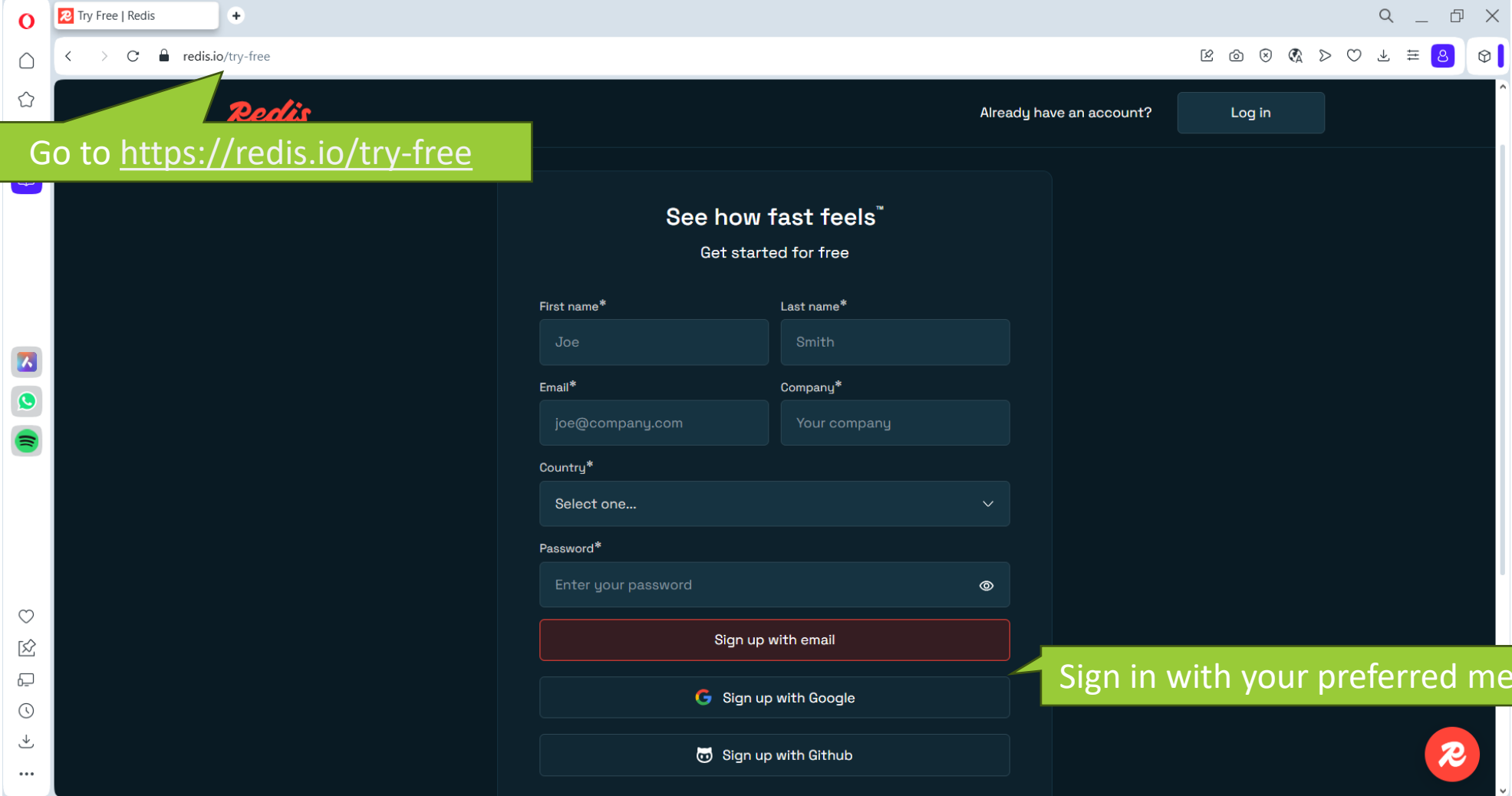
Anyone with a link to this project
Disabled

Learn more about sharing in [our docs](#).

Copy link

Turn this notebook into a powerful data app or an interactive dashboard.
[Learn more](#) [Create App](#)

Cloud Environment Setup: Redis



The screenshot shows the Redis 'Try Free' sign-up page in a web browser. The browser's address bar shows 'redis.io/try-free'. A green callout bubble points to the URL in the address bar with the text 'Go to <https://redis.io/try-free>'. The page has a dark blue background with the Redis logo in red at the top left. In the top right, there is a link 'Already have an account?' and a 'Log in' button. The main heading is 'See how fast feels™' followed by 'Get started for free'. The sign-up form includes fields for 'First name*' (containing 'Joe'), 'Last name*' (containing 'Smith'), 'Email*' (containing 'joe@company.com'), 'Company*' (containing 'Your company'), 'Country*' (a dropdown menu showing 'Select one...'), and 'Password*' (containing 'Enter your password'). Below these fields are three buttons: 'Sign up with email' (highlighted with a red border), 'Sign up with Google', and 'Sign up with Github'. A green callout bubble points to these buttons with the text 'Sign in with your preferred method'. The bottom right corner features the Redis logo.

Try Free | Redis

redis.io/try-free

Redis

Already have an account? Log in

See how fast feels™
Get started for free

First name* Last name*
Joe Smith

Email* Company*
joe@company.com Your company

Country*
Select one...

Password*
Enter your password

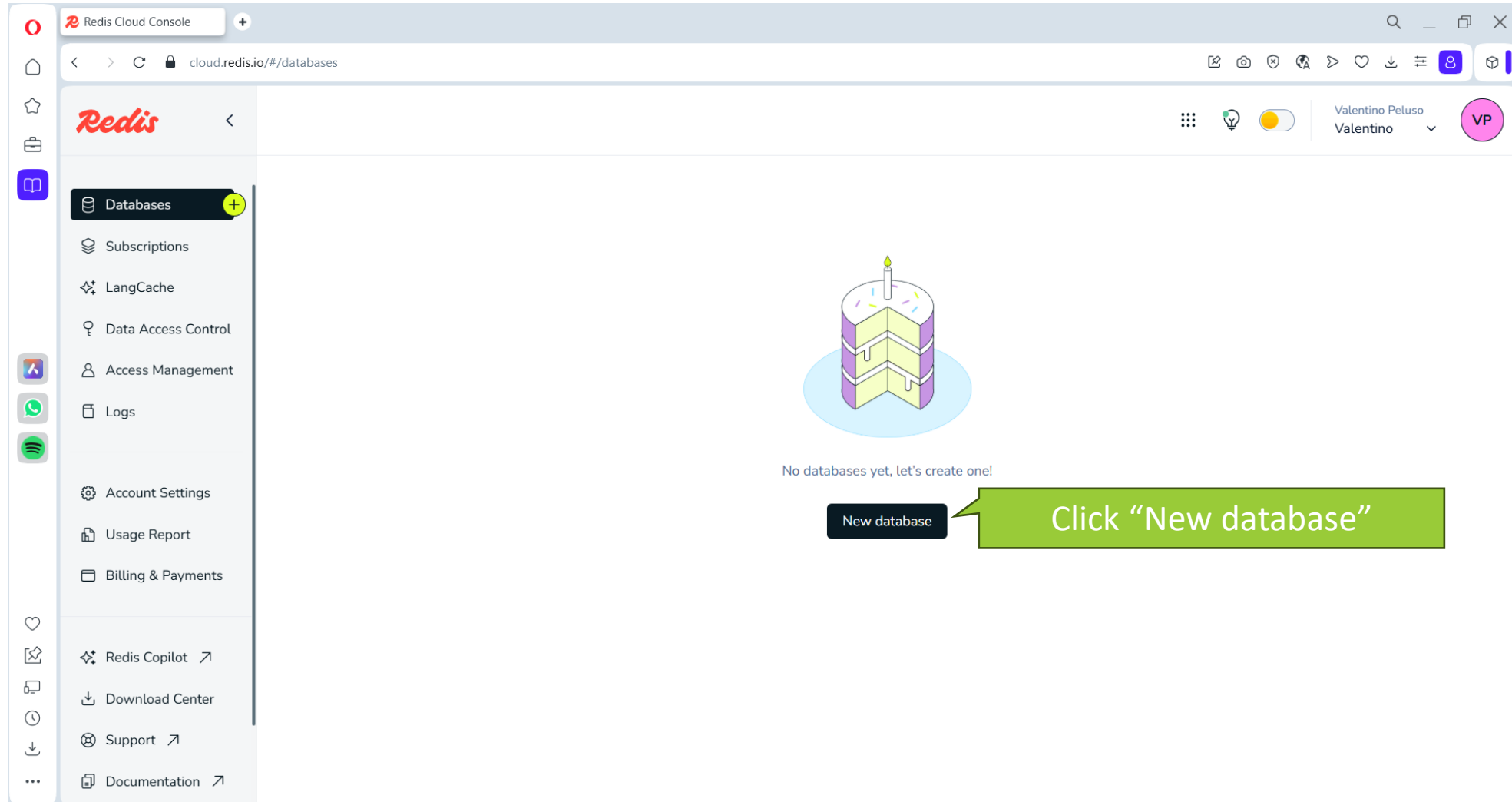
Sign up with email

Sign up with Google

Sign up with Github

Sign in with your preferred method

Cloud Environment Setup: Redis



Cloud Environment Setup: Redis

The screenshot displays the Redis Cloud Console interface. The left sidebar contains navigation links: Databases, Subscriptions, LangCache, Data Access Control, Access Management, Logs, Account Settings, Usage Report, Billing & Payments, Redis Copilot, Download Center, Support, and Documentation. The main content area is titled 'Create database' and shows the price '\$5/month'. Below this, a section titled 'What type of subscription do you need?' presents four options:

- Free**: Perfect for learning and exploring Redis. Always \$0. Up to 30 MB. Select plan.
- Flex**: Ideal for larger, cost-efficient workloads. From \$0.007/hour. Select plan.
- Essentials**: Scale up as your application grows. From \$0.007/hour. Select plan.
- Pro**: For teams building mission-critical systems in the cloud. From \$0.274/hour. Total \$200/month. Select plan.

A green callout box with the text 'Click "Select plan" under the "Free" box' points to the 'Select plan' button for the Free plan.

Plan	Description	Price	Features
Free	Perfect for learning and exploring Redis	Always \$0	Up to 30 MB, Shared cloud deployment, 30 MB single DB, SAML SSO, RBAC, encryption in transit, encryption at rest, Best effort SLA, community support
Flex	Ideal for larger, cost-efficient workloads	From \$0.007/hour	1-100 GB (RAM+SSD), single DB, SAML SSO, RBAC, encryption in transit, encryption at rest, 99.99% uptime, basic support only
Essentials	Scale up as your application grows	From \$0.007/hour	250 MB-12 GB RAM, single DB, SAML SSO, RBAC, encryption in transit, encryption at rest, 99.99% uptime, basic support only
Pro	For teams building mission-critical systems in the cloud	From \$0.274/hour, Total \$200/month	Dedicated cloud deployment, 6 GB RAM and up, multiple DBs, Includes Essentials, plus: Active-active (multi-region), auto-tiering, private connectivity, 99.999% uptime, 24/7 SLA-free support hotline, First \$200 free

Cloud Environment Setup: Redis

Redis Cloud Console

cloud.redis.io/#/add-subscription/free

Redis

Create database

Price \$0/month

Settings

Name: database-MGJ6DYIP

Database version: ☒ 7.4 ☐ 8.0 [Preview](#)

Cloud vendor: [aws](#) Amazon Web Services

Region: [Germany](#) Europe (Frankfurt) eu-central-1

Included in your Free plan

- ✓ 1 database
- ✓ 30 connections
- ✓ 30MB RAM
- ✓ Capabilities

1. Select "Europe (Frankfurt)" as the Region

2. Click "Create database"

Cancel Create database

Cloud Environment Setup: Redis

The screenshot displays the Redis Cloud Console interface. The left sidebar contains navigation links for Databases, Subscriptions, LangCache, Data Access Control, Access Management, Logs, Account Settings, Usage Report, Billing & Payments, Redis Copilot, Download Center, Support, and Documentation. The main content area shows the configuration for a specific database. At the top, there's a breadcrumb trail: 'All Databases' > 'database-MGJ6DYIP' > 'Database #13627908'. Below this, there are tabs for 'Configuration', 'Metrics', and 'Slowlog'. The 'Configuration' tab is active, showing a 'General' section with fields for 'Database name' (database-MGJ6DYIP) and 'Subscription' (database-MGJ6DYIP). A 'Public endpoint' is listed as 'redis-12253.crce198.eu-central-1-3.ec2.redns.redis-cloud.com:12253'. A green callout box points to this endpoint with the text: 'Get the "Public Endpoint" under the "General" box'. At the bottom of the configuration section, there's a 'Connect' button and a link to 'Connect using Redis CLI, Client, or Insight'.

Redis Cloud Console

cloud.redis.io/#/databases/13627908/subscription/2942362/view-bdb/configuration

Redis

Databases

Subscriptions

LangCache

Data Access Control

Access Management

Logs

Account Settings

Usage Report

Billing & Payments

Redis Copilot

Download Center

Support

Documentation

All Databases

database-MGJ6DYIP

Database #13627908

Configuration Metrics Slowlog

Import Connect Launch

General

Database name

database-MGJ6DYIP

Subscription

database-MGJ6DYIP

Public endpoint

redis-12253.crce198.eu-central-1-3.ec2.redns.redis-cloud.com:12253

Connect using Redis CLI, Client, or Insight

Connect

Get the "Public Endpoint" under the "General" box

Cloud Environment Setup: Redis

The screenshot displays the Redis Cloud Console interface. The left sidebar contains navigation links for Databases, Subscriptions, LangCache, Data Access Control, Access Management, Logs, Account Settings, Usage Report, Billing & Payments, Redis Copilot, Download Center, Support, and Documentation. The main content area shows the configuration for 'database-MGJ6DYIP' (Database #13627908). The 'Configuration' tab is active, showing sections for Security, Alerts, and Danger zone. A green callout box with the text 'Expand the "Security" box' points to the 'Security' section header. The 'Security' section shows 'Default user' as 'On'. The 'Alerts' section shows 'Active alerts' as '2'. The 'Danger zone' section contains a 'Delete database' button and a warning message: 'Permanently delete the database and its contents'.

Cloud Environment Setup: Redis

The screenshot displays the Redis Cloud Console interface. The left sidebar contains navigation links for Databases, Subscriptions, LangCache, Data Access Control, Access Management, Logs, Account Settings, Usage Report, Billing & Payments, Redis Copilot, Download Center, Support, and Documentation. The main content area shows the configuration for a specific database, 'database-MGJ6DYIP'. The 'Security' section is expanded, revealing the 'Default user' settings. The 'Default user password' field is masked with dots, and a 'Copy' button is visible next to it. A green callout box with the text 'Copy the password' points to this 'Copy' button. Other sections visible include 'Alerts' and 'SlowLog'.

Clone the “Redis Connection” Notebook

The screenshot shows the Deepnote web interface. At the top, the browser address bar displays the URL: `deepnote.com/workspace/ECAI-26-e30878e0-87e9-4b7c-b044-c52c3f4dd72f/project/My-Private-Project-94727b9a-1607-4a9c-a3d1-798fd357db4d/notebook/7298453df6074ae`. The interface includes a sidebar on the left with sections for Notebooks, Integrations, Files, and Machine. The main area displays a notebook titled "Notebook 1" containing Python code that checks the system's Python and PyTorch versions. A green callout box with the text "Click the Home icon" points to the Home icon in the top navigation bar. The code in the notebook is as follows:

```
1 import sys
2 import torch
3
4 print(sys.version)
5 if sys.version.split()[0] != '3.12.7':
6     raise RuntimeError('Wrong Python version. Expected Version is 3.12.7.')
7
8 print('Python version: OK\n')
9
10 print(torch.__version__)
11 if torch.__version__ != '2.7.0+cu126':
12     raise RuntimeError('Wrong PyTorch version. Expected Version is 2.7.0+cu126.')
13
14 print('PyTorch version: OK')
```

The output of the code is displayed below the code block:

```
3.12.7 (main, Sep 11 2025, 12:40:23) [GCC 10.2.1 20210110]
Python version: OK

2.7.0+cu126
PyTorch version: OK
```

The bottom of the interface shows a toolbar with options for Code, Text, SQL, Chart, Input, and Module. The URL at the bottom of the page is `https://deepnote.com/workspace/ECAI-26-e30878e0-87e9-4b7c-b044-c52c3f4dd72f/private`.

Clone the “Redis Connection” Notebook

The screenshot shows the Redis Cloud Console interface. The left sidebar contains a navigation menu with the following items: Home, Integrations, Settings & members, Favorites, Recent projects, Private projects (1), Shared with me (2), Modules, Data apps, Scheduled projects, Projects (+), Live Coding, and Material (...). A green callout box points to the 'Material' option under the 'Projects' section, with the text "Click 'Material' under 'Projects'". The main content area displays the 'Private projects' page, which includes a search bar, a 'Create new' button, and a table of projects.

Name	Status	Sharing	Author	Created	Last opened by me
My Private Project	Idle	Locked	Valentino Peluso T...	32 minutes ago	Opened 1 minute ago

The URL bar shows the workspace ID: `deepnote.com/workspace/ECAI-26-e30878e0-87e9-4b7c-b044-c52c3f4dd72f/private`. The bottom status bar shows the project ID: `https://deepnote.com/workspace/ECAI-26-e30878e0-87e9-4b7c-b044-c52c3f4dd72f/project/Material-3880e510-b64c-4bb5-b488-c2122d5d9e2d`.

Clone the “Redis Connection” Notebook

The screenshot shows the Deepnote web interface. On the left sidebar, under the 'Notebooks' section, the notebook '3.1 - Lab1 - Redis Connection' is selected. A green callout box with a pointer to the three-dot menu next to this notebook contains the following instructions:

1. Click “...” next to “3.1 – Lab1 – Redis Connection”
2. Click “Download as .ipynb”

The main content area displays the notebook's title 'Redis Connection' and the section 'Import the required modules'. Below this, there is a code editor showing the following Python code:

```
1 redis_client = redis.Redis(  
2     host=REDIS_HOST,  
3     port=REDIS_PORT,
```

Clone the “Redis Connection” Notebook

The screenshot shows the Deepnote web interface. On the left sidebar, the 'Files' section is expanded, and a dropdown menu is open with 'Upload .ipynb file' selected. A green callout box with white text provides instructions: 1. Go back to your private project, 2. Click “+” next to “Files”, 3. Click “Upload .ipynb file”, 4. Select the downloaded notebook. An arrow points from the fourth step to a file explorer window on the right. The file explorer shows a list of files, with '3.1 - Lab1 - Redis Connection.ipynb' highlighted. The background interface shows a notebook editor with Python code and error messages.

1. Go back to your private project
2. Click “+” next to “Files”
3. Click “Upload .ipynb file”
4. Select the downloaded notebook

ECAI

LAB1

Clone the “Redis Connection” Notebook

The screenshot shows the Redis Cloud Console interface. On the left sidebar, there's a 'Notebooks' section with 'Init' and '3.1 - Lab1 - Redis Connection' (selected). Below it is 'Integrations' and 'Files'. The 'Machine' section shows 'CPU (Basic)' with '2 vCPU, 5 GB memory' and a 'Dockerfile' dropdown, with a 'Start machine' button.

The main area displays the notebook 'Redis Connection' with the title 'Import the required modules'. The code editor shows:

```
1 import redis
```

Below this, a code block contains the following configuration:

```
1 REDIS_HOST = 'redis-12253.crce198.eu-central-1-3.ec2.redns.redis-cloud.com'
2 REDIS_PORT = 12253
3 REDIS_USERNAME = 'default'
4 REDIS_PASSWORD = 'd2mRknSH4LqfBhUZzQ6ZiYUHjStR2LTy'
```

Two green callout boxes provide instructions:

- A box pointing to the host string: "E.g.: **redis-12253.crce198.eu-central-1-3.ec2.redns.redis-cloud.com:12253**"
- A box pointing to the port value: "Remove “:<port>” in REDIS_HOST"
- A box pointing to the port value: "REDIS_PORT must be *int* (not *str*)"

Below the code block, the section 'Connect to the DB' is visible with the following code:

```
1 redis_client = redis.Redis(
2     host=REDIS_HOST
```

Clone the “Redis Connection” Notebook

The screenshot shows the Deepnote web interface. On the left sidebar, the 'Notebooks' section is active, showing a notebook titled '3.1 - Lab1 - Redis Connection'. The main workspace displays three code blocks:

- Connect to the DB**: Contains Python code to connect to a Redis database. A green callout box with the text 'Click “Run”' points to the 'Run' button above the code.
- Create a new entry**: Contains Python code to set a Redis key 'message' to 'Hello World!'. A green callout box with the text 'Check the expected output' points to the output 'Message written: True'.
- Read the entry value**: Contains Python code to get the Redis key 'message'. A green callout box with the text 'Check the expected output' points to the output 'Hello World!'.

The bottom of the interface shows a 'Machine' section with a 'CPU (Basic)' configuration and a 'Dockerfile' button. The bottom right corner has a 'Module' dropdown menu.

Local Environment Setup: RPI

The provided SD card mounts a development environment running on Raspbian OS, including Python 3.11 and all necessary Python packages for the LABs.



1. Connect the RPI to your laptop using the Ethernet cable



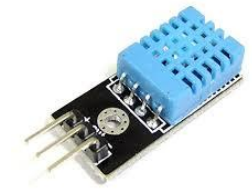
2. Connect the USB microphone to an RPI USB port



Local Environment Setup: RPI

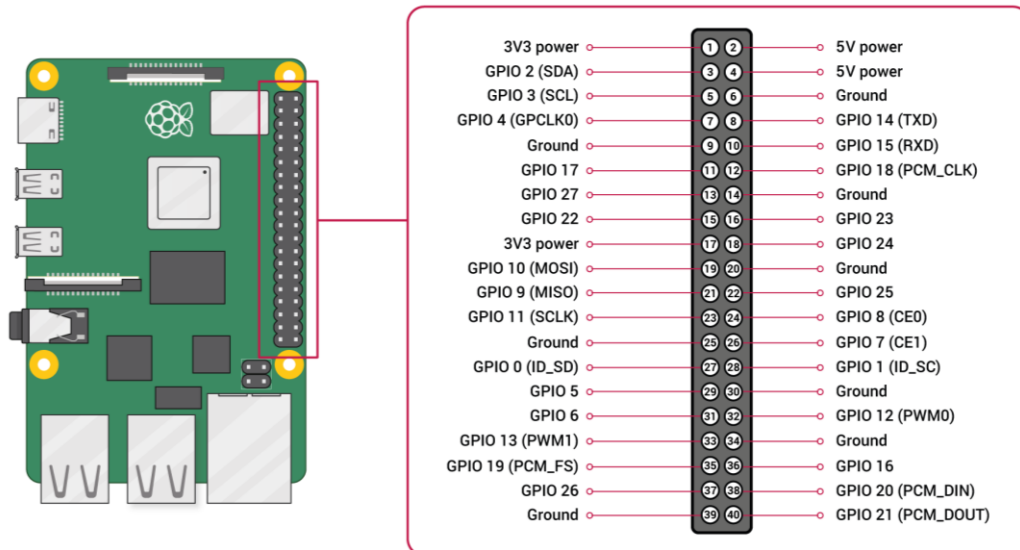
3. Connect the DHT-11 sensor to the RPI (refer to the schematic below):

- Connect the **VCC** (+) pin of DHT-11 to the **3V3** pin of RPI (pin 1 in the figure)
- Connect the **DATA** (out) pin of DHT-11 to the **GPIO 4** pin of RPI (pin 7)
- Connect the **GND** (-) pin of DHT-11 to the **Ground** pin of RPI (pin 6)

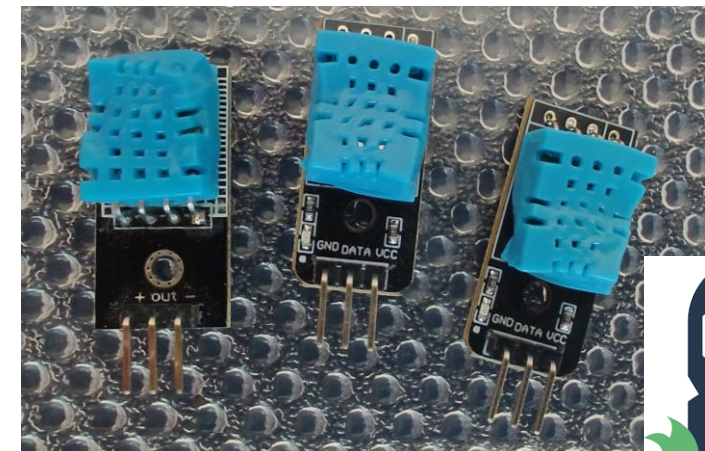


WARNING: Wrong connections will damage the sensor!

4. Connect the RPI to the power supply (wait approx. 2 minutes for the startup)



LAB1



**Fried DHT-11 sensors from last year
due to wrong connection**



Local Environment Setup: RPI



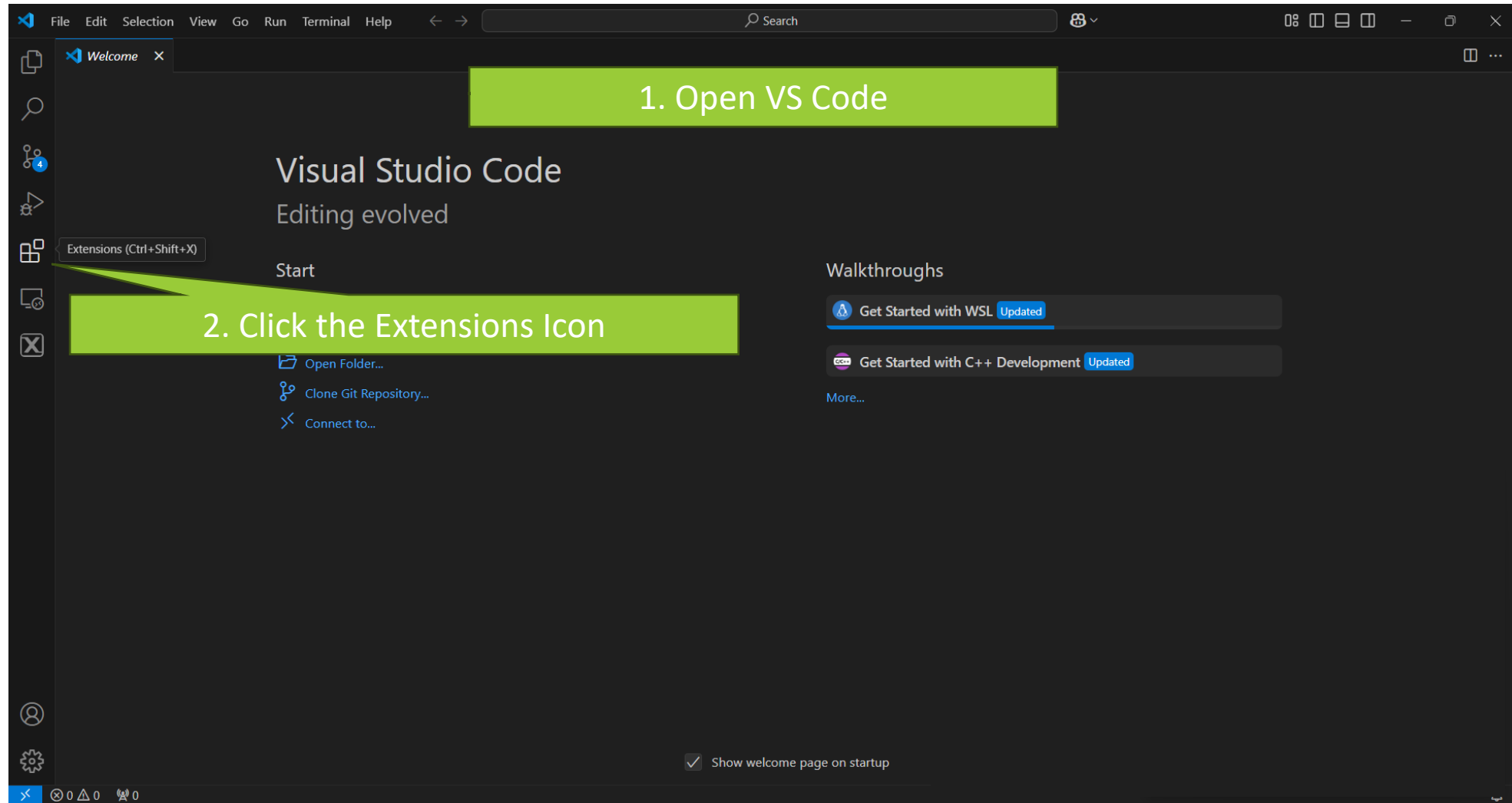
WARNING



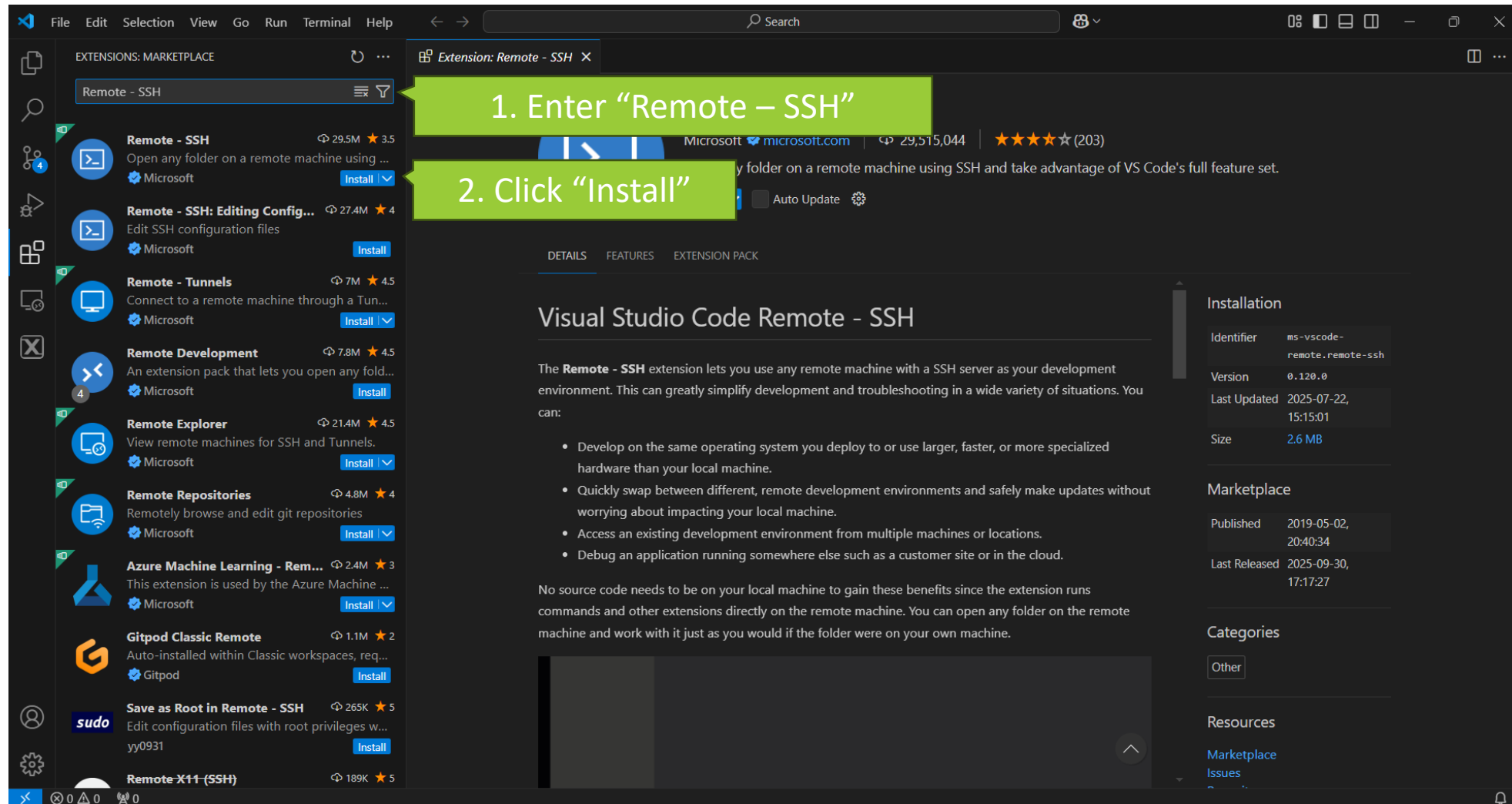
Before disconnecting the RPI from the power supply, shutdown the RPI using the command **sudo shutdown -h now** and disconnect the power supply only after the green LED stops blinking.

Failing to do so could damage the RPI and/or SD card!!!

Local Environment Setup: VS Code



Local Environment Setup: VS Code



Local Environment Setup: SSH into RPI

The screenshot shows the Visual Studio Code interface. On the left, the 'View' menu is open, highlighting 'Command Palette...' (Ctrl+Shift+P). A green callout box with a terminal icon contains the following instructions:

1. Click "View"
2. Click "Command Palette..."

The main panel displays the 'Visual Studio Code Remote - SSH' extension page. The extension is by Microsoft, with 29,515,044 downloads and a 4.5-star rating (203 reviews). It is described as: 'Open any folder on a remote machine using SSH and take advantage of VS Code's full feature set.' The page includes sections for 'DETAILS', 'FEATURES', and 'EXTENSION PACK'. The 'FEATURES' section lists:

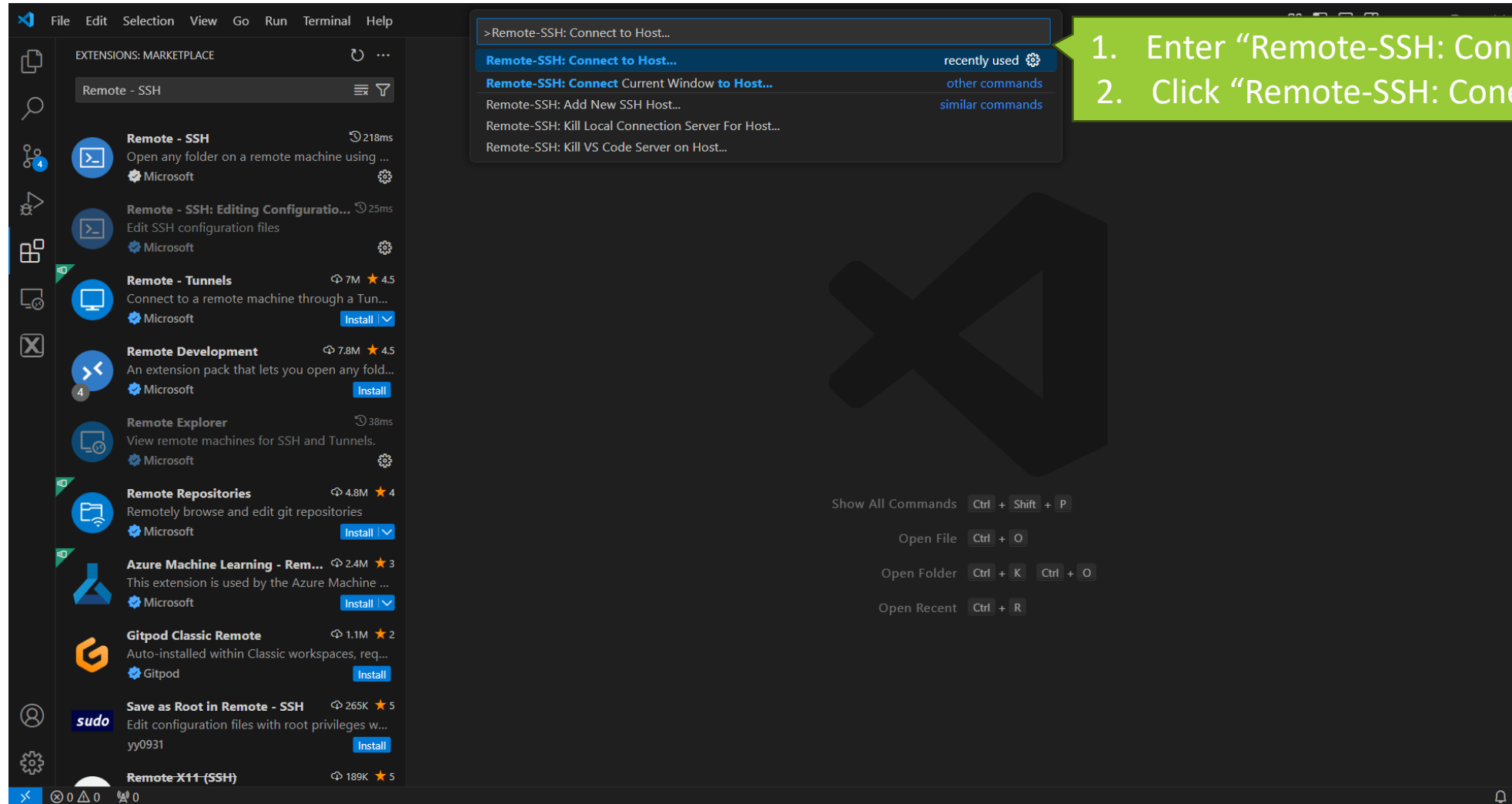
- Develop on the same operating system you deploy to or use larger, faster, or more specialized hardware than your local machine.
- Quickly swap between different, remote development environments and safely make updates without worrying about impacting your local machine.
- Access an existing development environment from multiple machines or locations.
- Debug an application running somewhere else such as a customer site or in the cloud.

Below the features, it states: 'No source code needs to be on your local machine to gain these benefits since the extension runs commands and other extensions directly on the remote machine. You can open any folder on the remote machine and work with it just as you would if the folder were on your own machine.'

The 'Installation' section shows the extension's identifier as 'ms-vscode-remote.remote-ssh', version '0.120.0', last updated '2025-07-22, 15:15:01', and size '2.6 MB'. The 'Marketplace' section shows it was published on '2019-05-02, 20:40:34' and last released on '2025-09-30, 17:17:27'. The 'Categories' section lists 'Other'. The 'Resources' section includes links to 'Marketplace' and 'Issues'.

The bottom of the screenshot shows a preview of the extension in use, with a terminal window open in a remote environment, displaying a Python script that imports 'time' and 'sklearn' modules and prints the start time.

Local Environment Setup: SSH into RPI



Local Environment Setup: SSH into RPI

1. Enter: `ecai@raspberrypi.local`
2. Press *Enter*

Visual Studio Code Remote - SSH

The **Remote - SSH** extension lets you use any remote machine with a SSH server as your development environment. This can greatly simplify development and troubleshooting in a wide variety of situations. You can:

- Develop on the same operating system you deploy to or use larger, faster, or more specialized hardware than your local machine.
- Quickly swap between different, remote development environments and safely make updates without worrying about impacting your local machine.
- Access an existing development environment from multiple machines or locations.
- Debug an application running somewhere else such as a customer site or in the cloud.

No source code needs to be on your local machine to gain these benefits since the extension runs commands and other extensions directly on the remote machine. You can open any folder on the remote machine and work with it just as you would if the folder were on your own machine.

Status	Message
Canceled	Error: Canceled (Connecting was canceled)

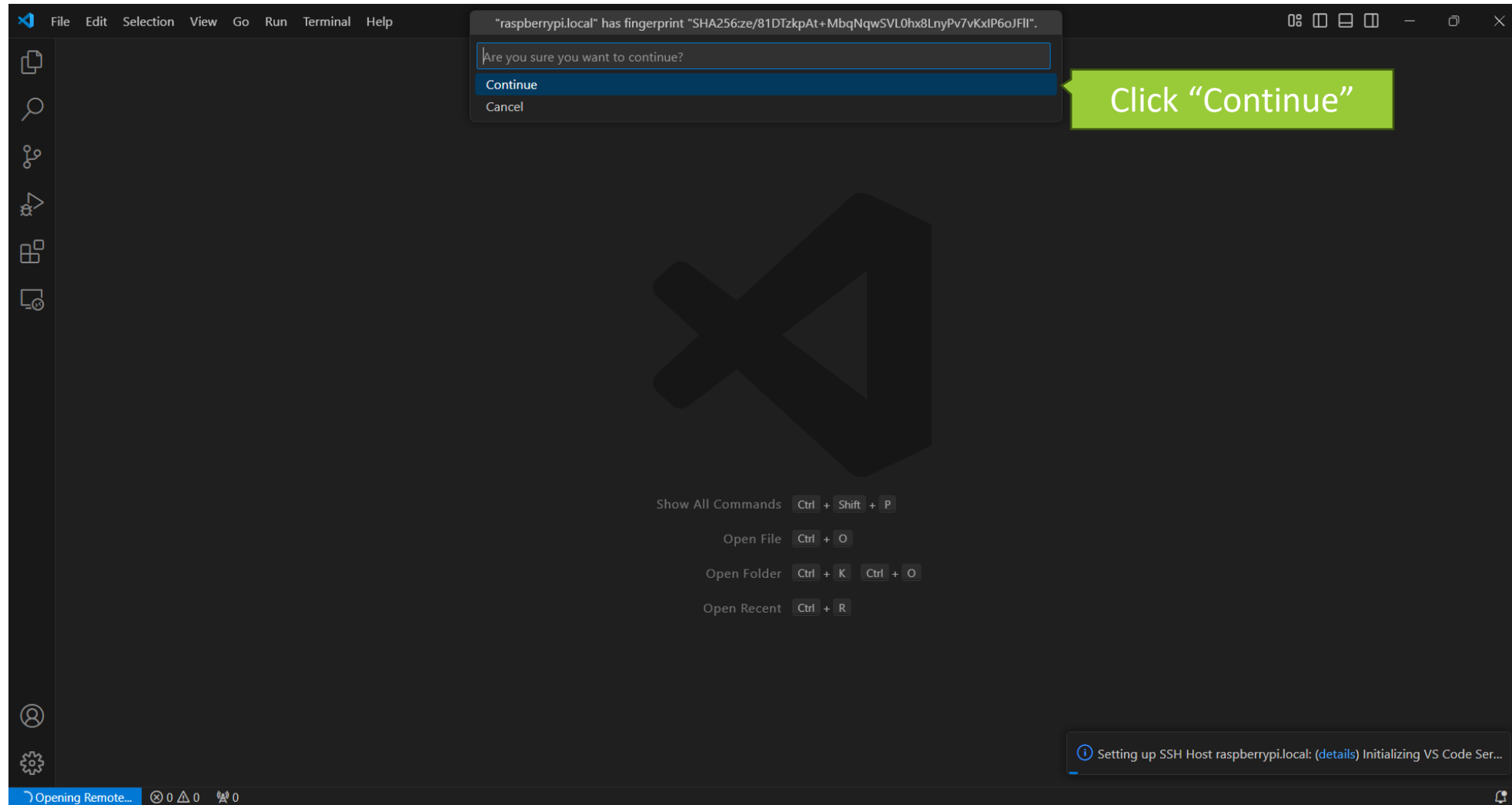
See verbose information in the [Output Log...](#)

Extensions installed: debug-tracker-vscode, Italian Language Pack for Visual Studio Co..., Jupyter Keymap, Memory Inspector, Remote - SSH, Remote - SSH: Editing Configuratio..., Remote Explorer, Docker, GitHub Copilot.

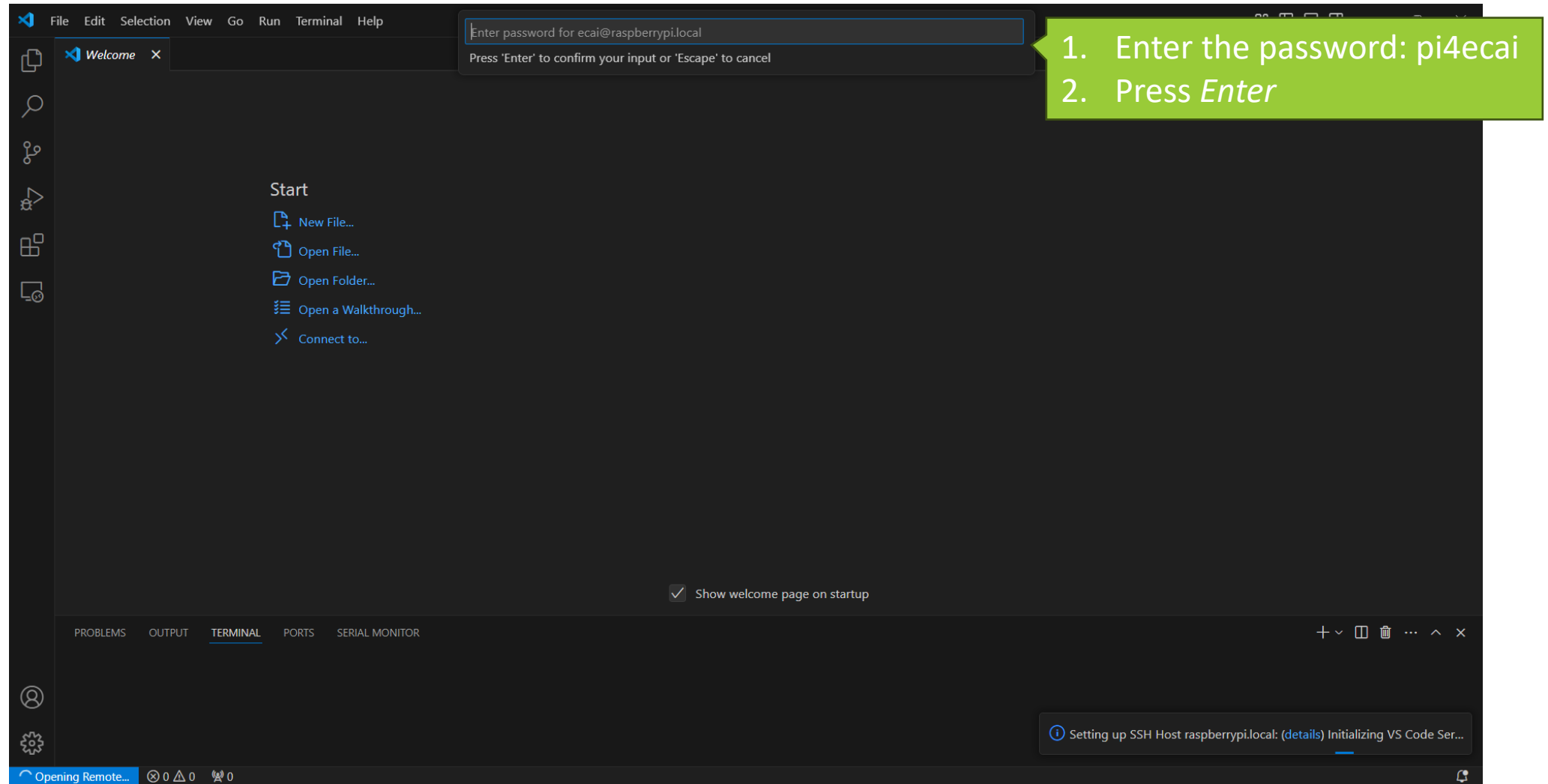
SSH - INSTALLED: 0
RECOMMENDED: 6

Disconnected from SSH

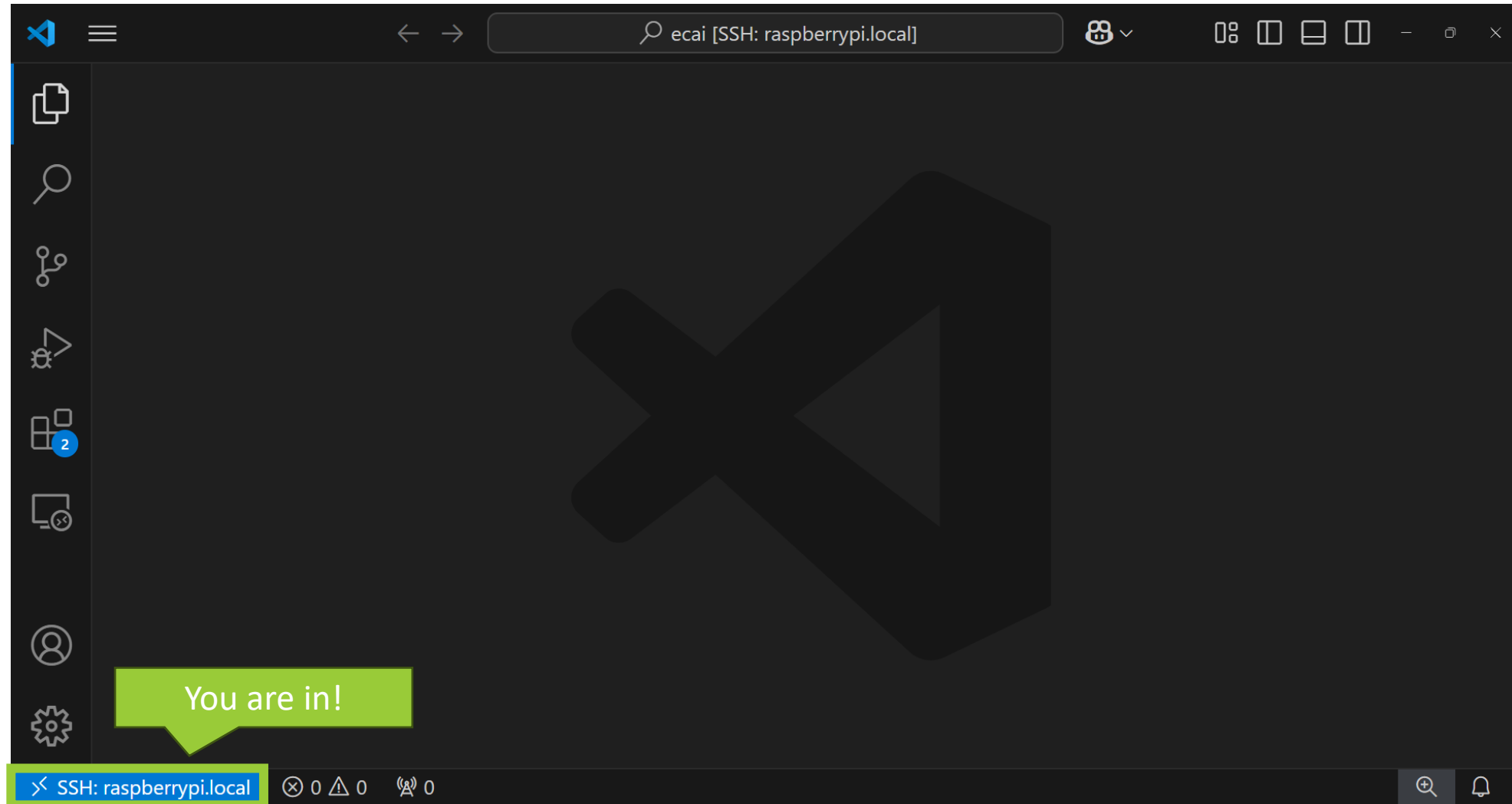
Local Environment Setup: SSH into RPI



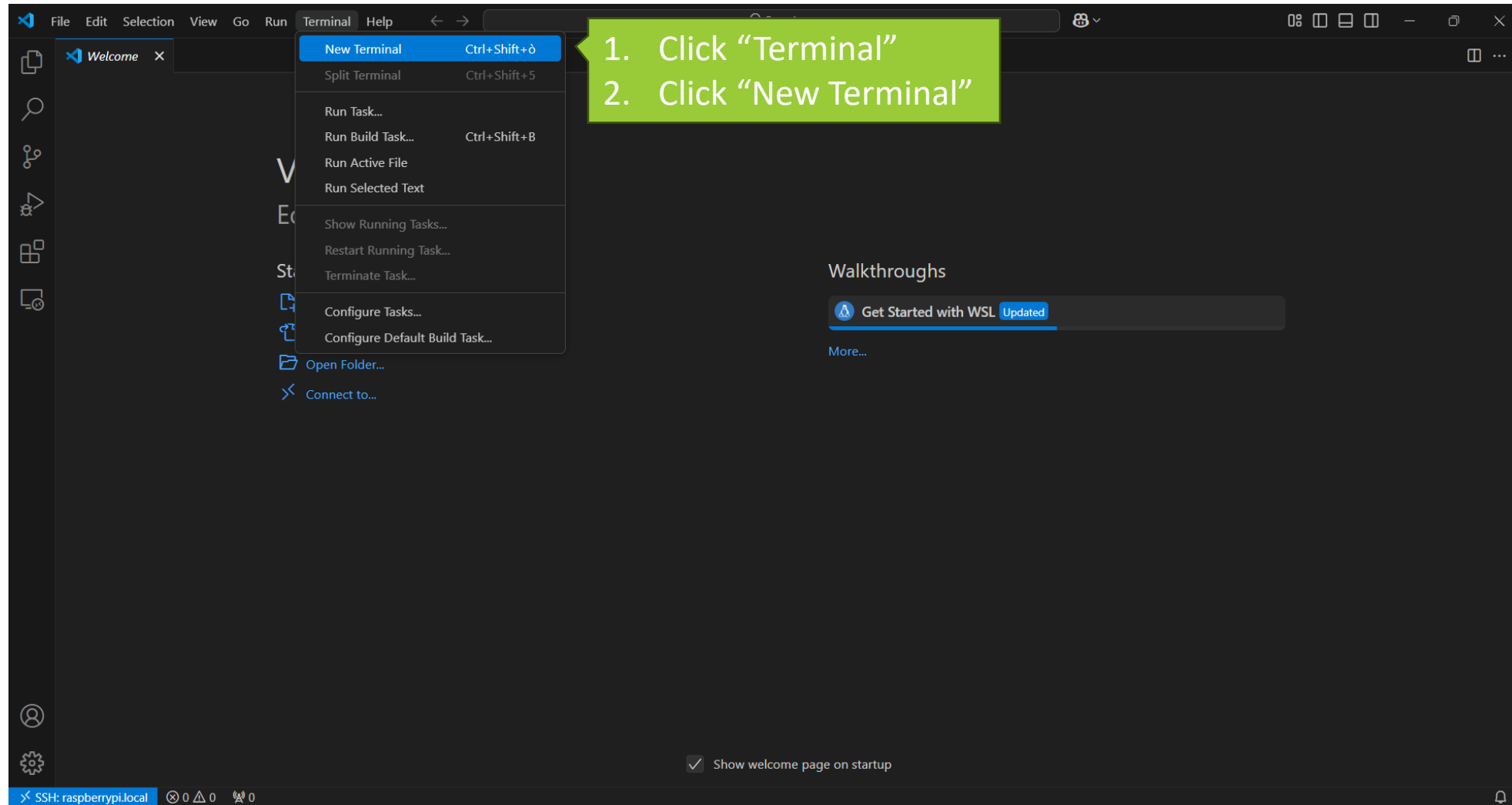
Local Environment Setup: SSH into RPI



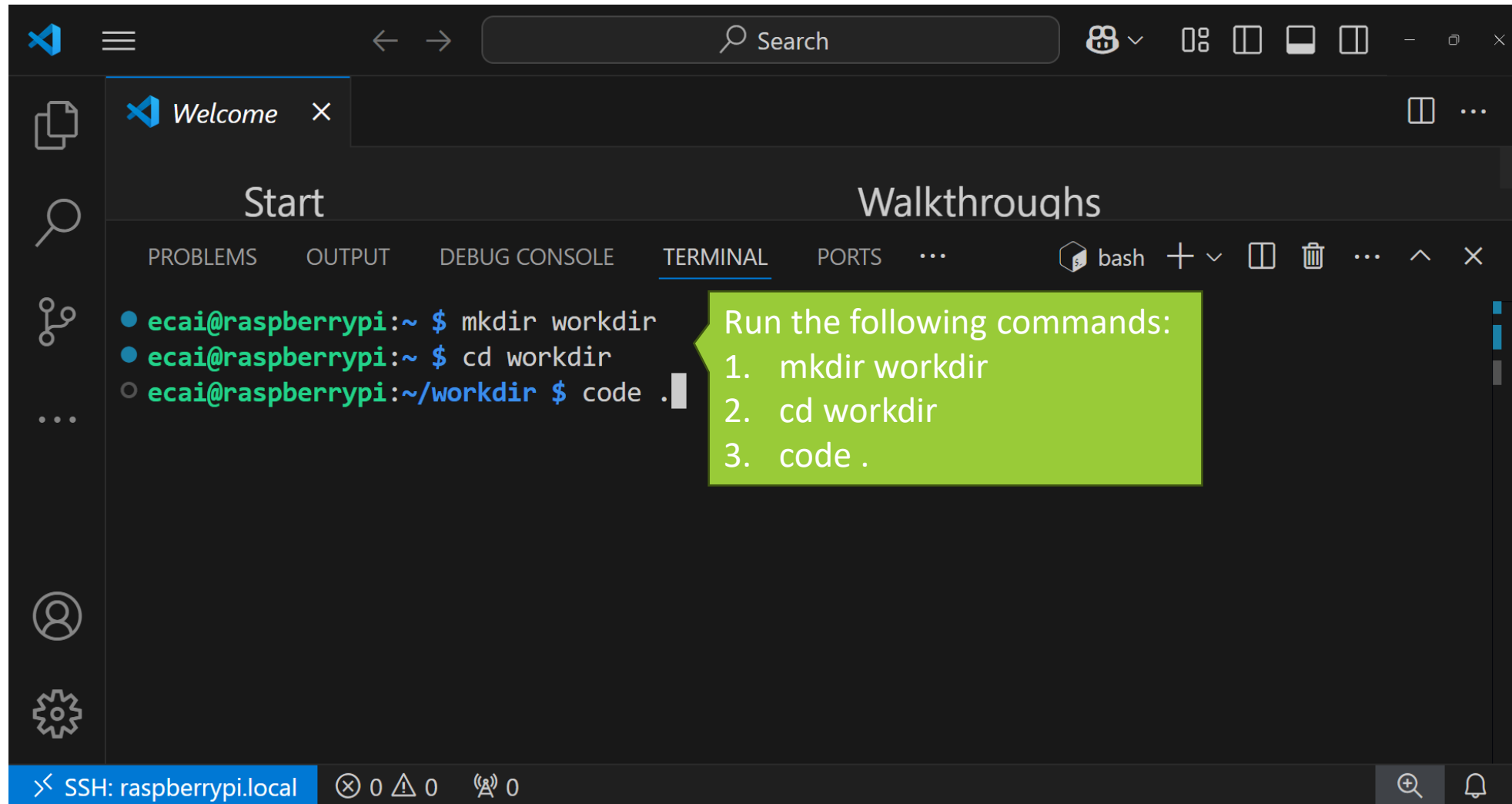
Local Environment Setup: SSH into RPI



Local Env. Setup: Create a Working Dir



Local Env. Setup: Create a Working Dir



The screenshot shows the Visual Studio Code interface. The terminal window is active, displaying the following commands and output:

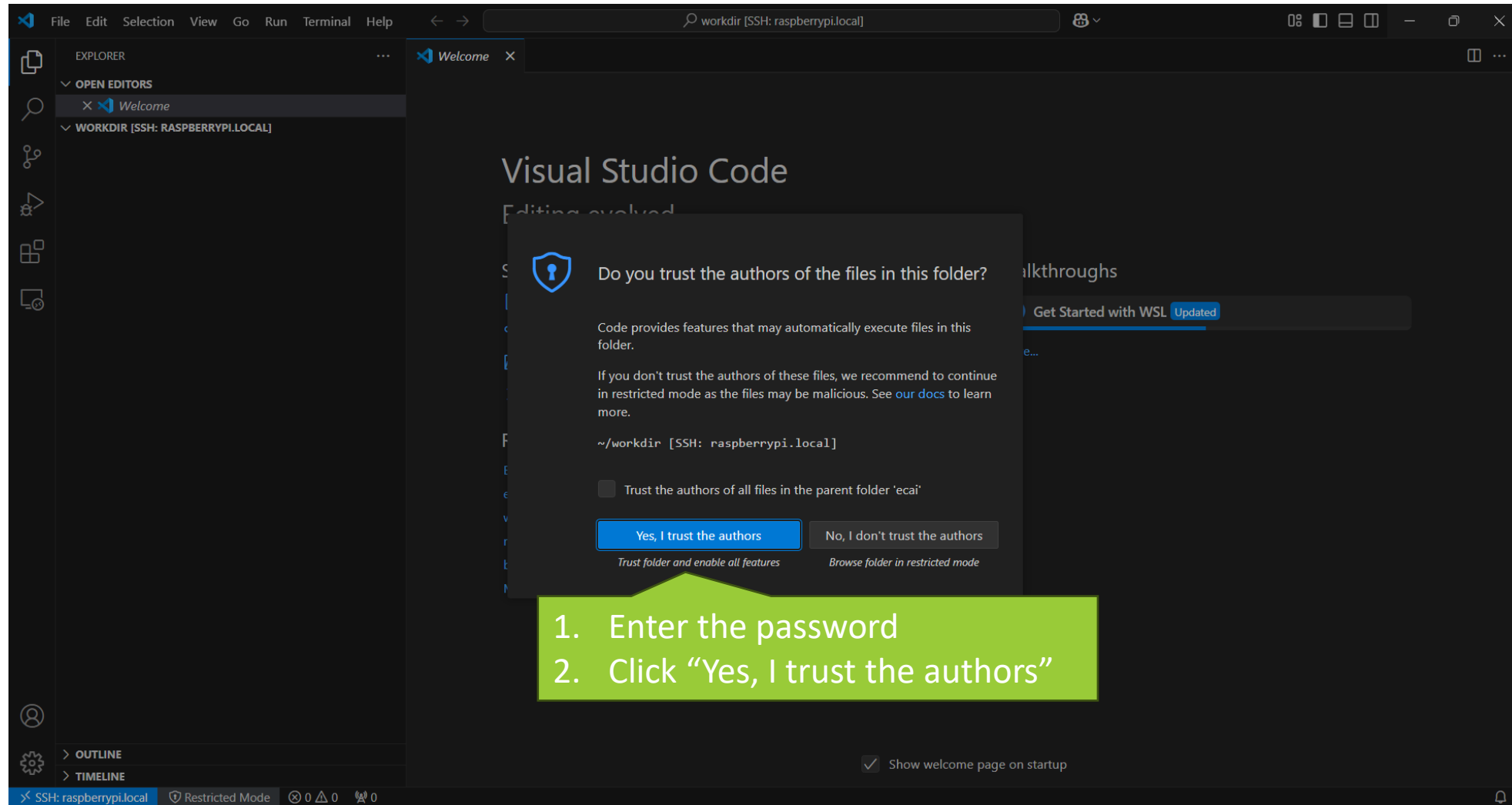
```
ecai@raspberrypi:~ $ mkdir workdir
ecai@raspberrypi:~ $ cd workdir
ecai@raspberrypi:~/workdir $ code .
```

A green callout box with the text "Run the following commands:" contains a numbered list:

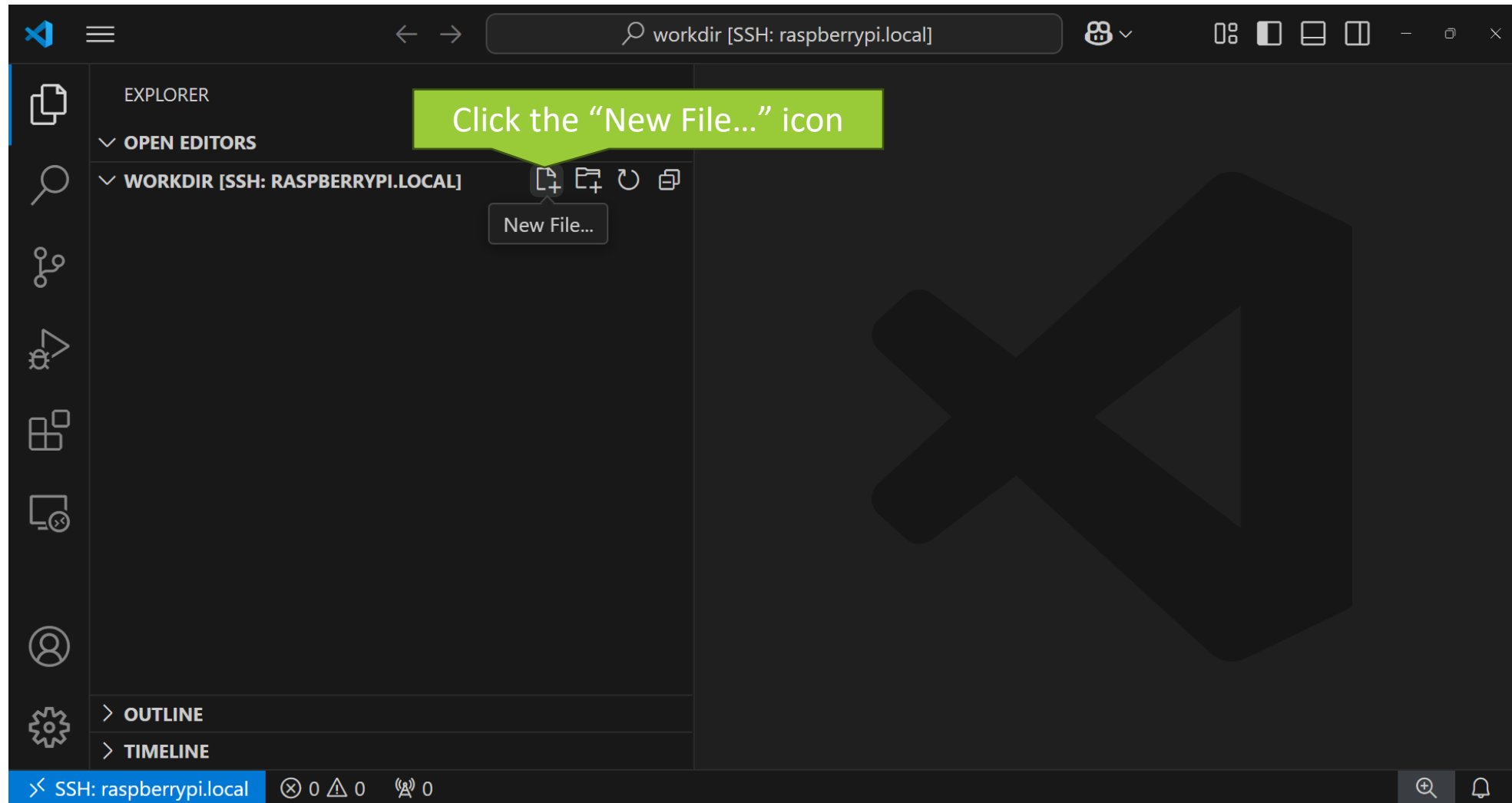
1. mkdir workdir
2. cd workdir
3. code .

The status bar at the bottom indicates the connection is "SSH: raspberrypi.local".

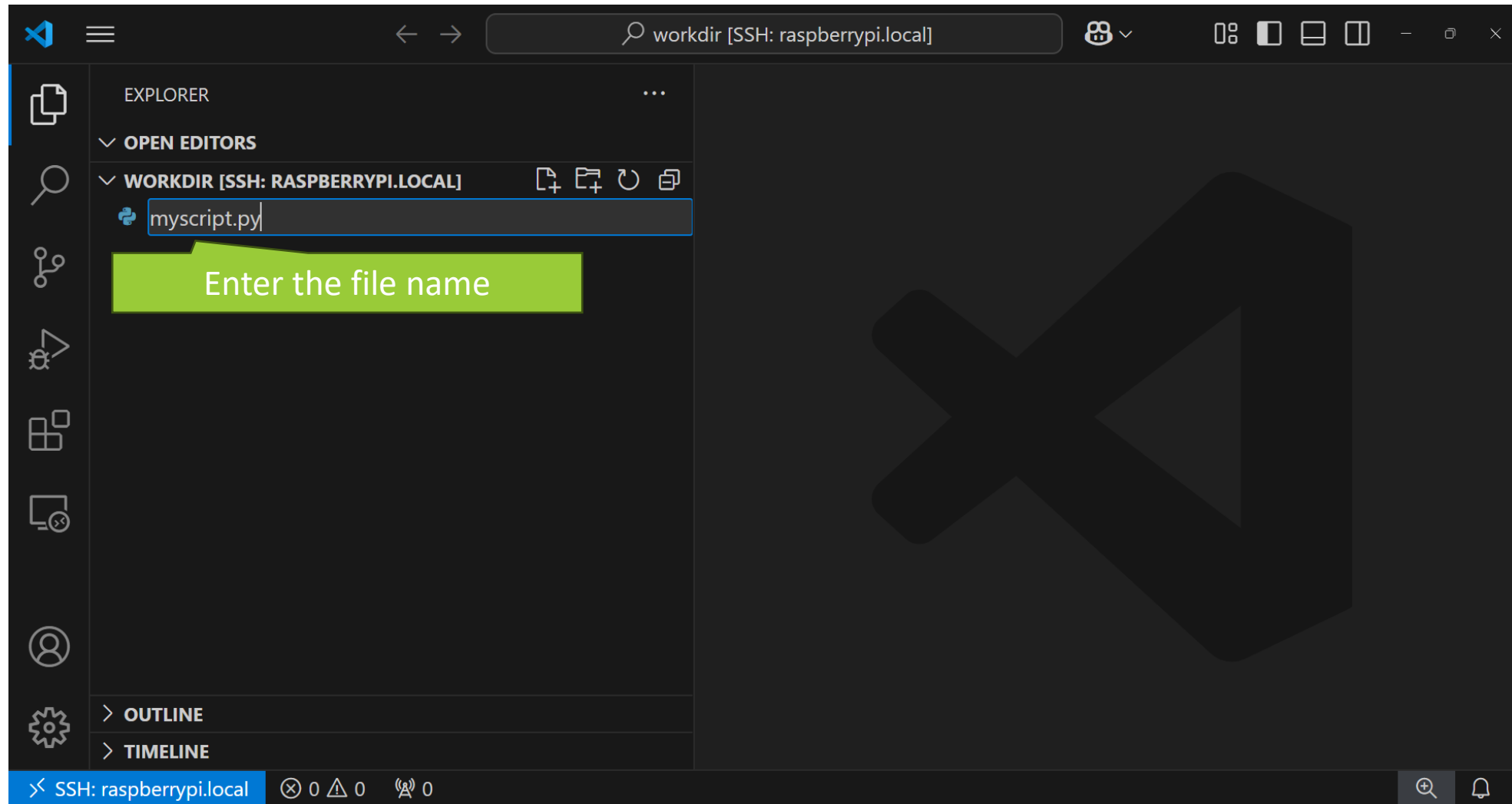
Local Env. Setup: Create a Working Dir



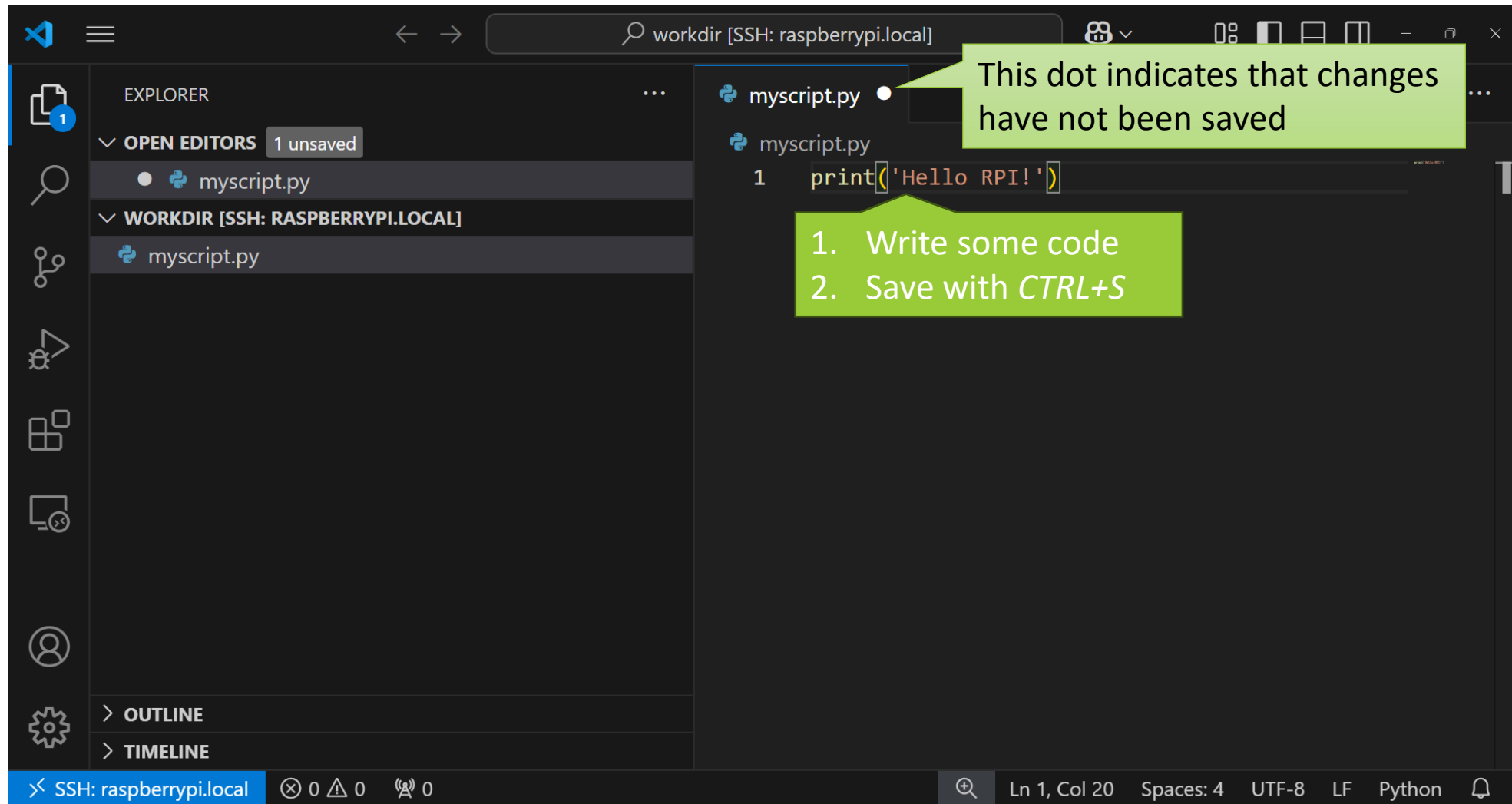
Local Env. Setup: Run your first script



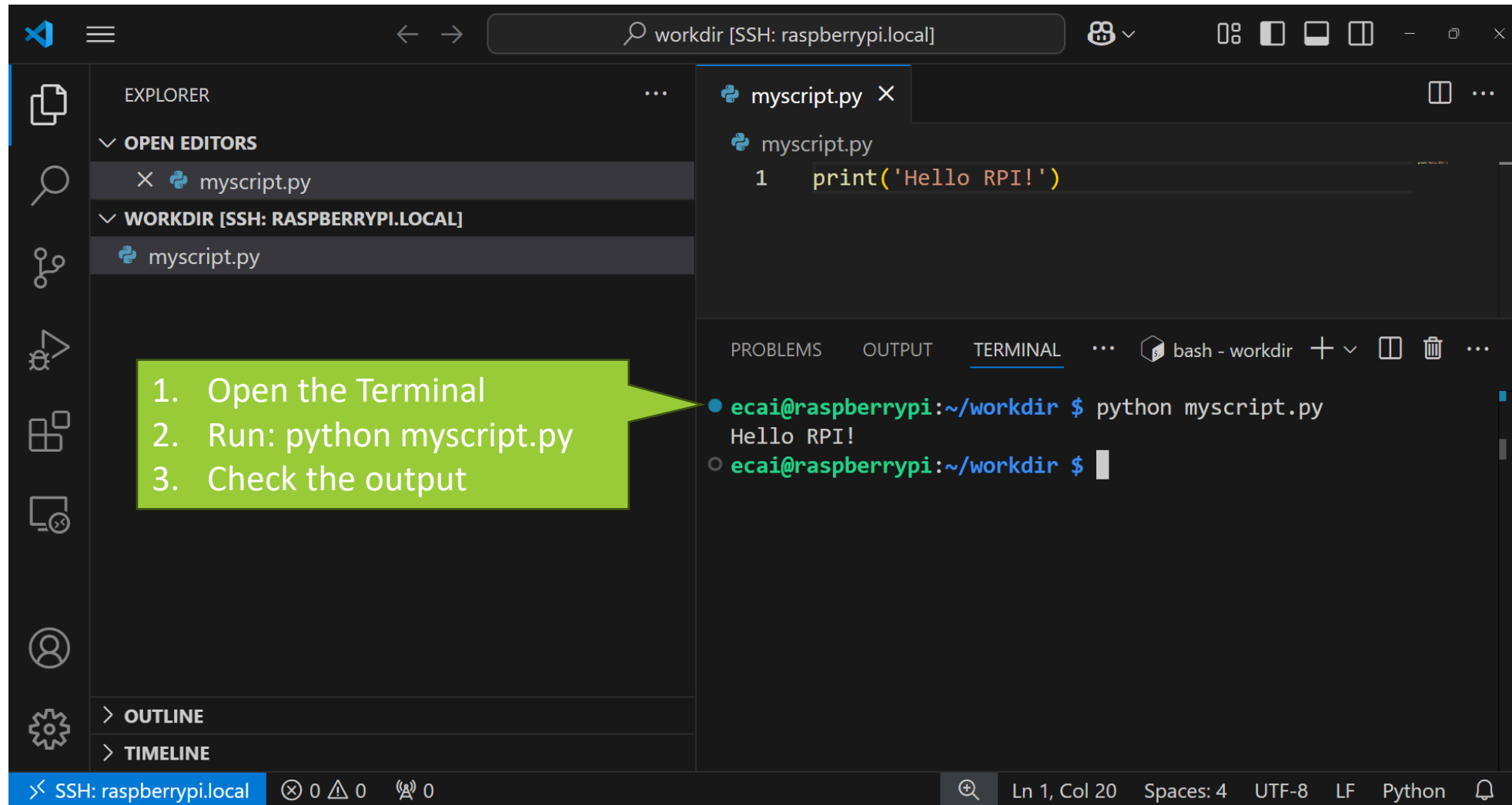
Local Env. Setup: Run your first script



Local Env. Setup: Run your first script



Local Env. Setup: Run your first script

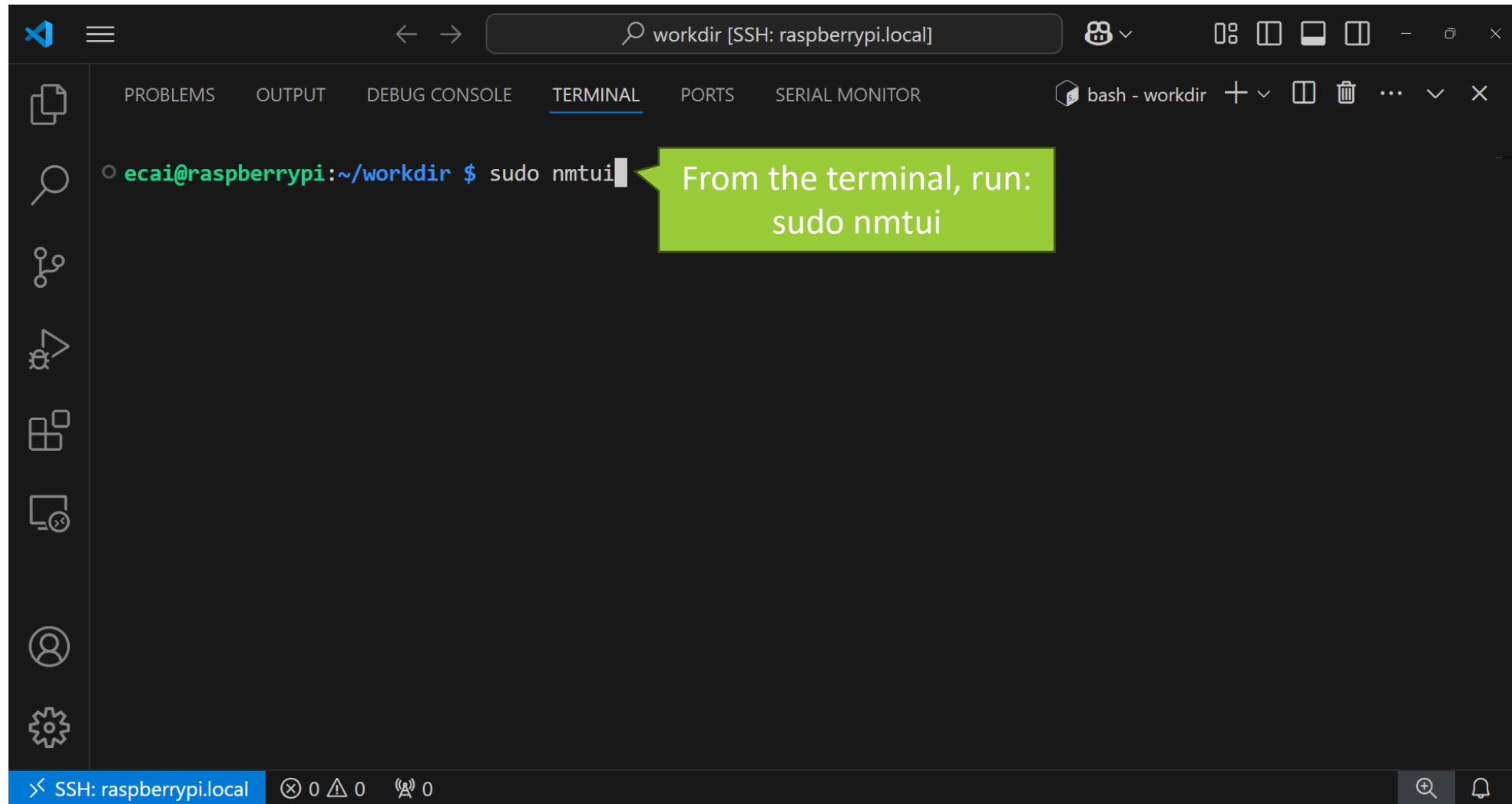


The screenshot shows the Visual Studio Code (VS Code) interface connected to a Raspberry Pi via SSH. The Explorer panel on the left shows the file structure with 'myscript.py' open in the editor. The terminal panel at the bottom shows the command 'python myscript.py' being executed, resulting in the output 'Hello RPI!'. A green callout box with a pointer to the terminal contains the following instructions:

1. Open the Terminal
2. Run: `python myscript.py`
3. Check the output

The status bar at the bottom indicates the current file is 'SSH: raspberrypi.local', line 1, column 20, with 4 spaces, UTF-8 encoding, LF line endings, and Python language mode.

Local Env. Setup: PoliTO WiFi

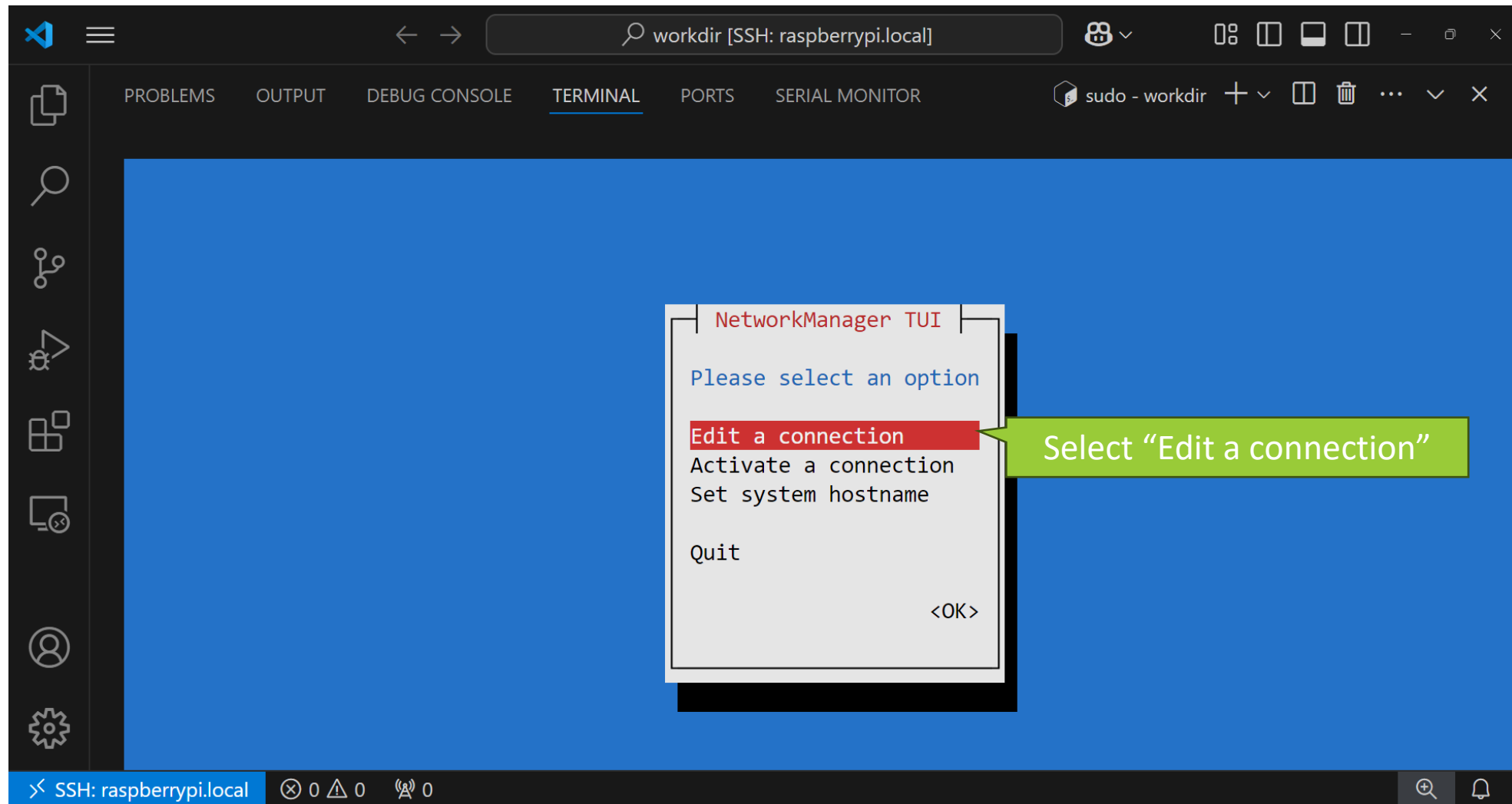


The image shows a Visual Studio Code interface with a terminal window open. The terminal title is 'workdir [SSH: raspberrypi.local]'. The terminal content shows the prompt 'ecai@raspberrypi:~/workdir \$' followed by the command 'sudo nmtui'. A green callout box with a pointer to the command contains the text 'From the terminal, run: sudo nmtui'. The terminal window has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, PORTS, and SERIAL MONITOR. The bottom status bar shows 'SSH: raspberrypi.local' and connection statistics.

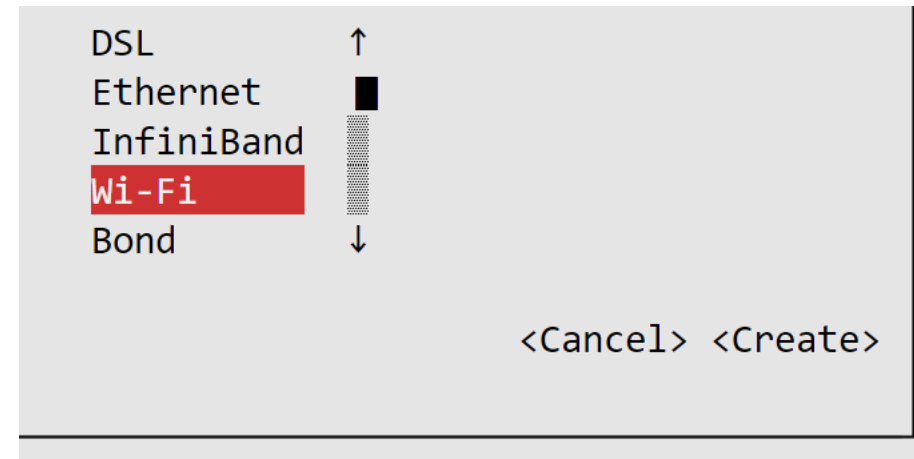
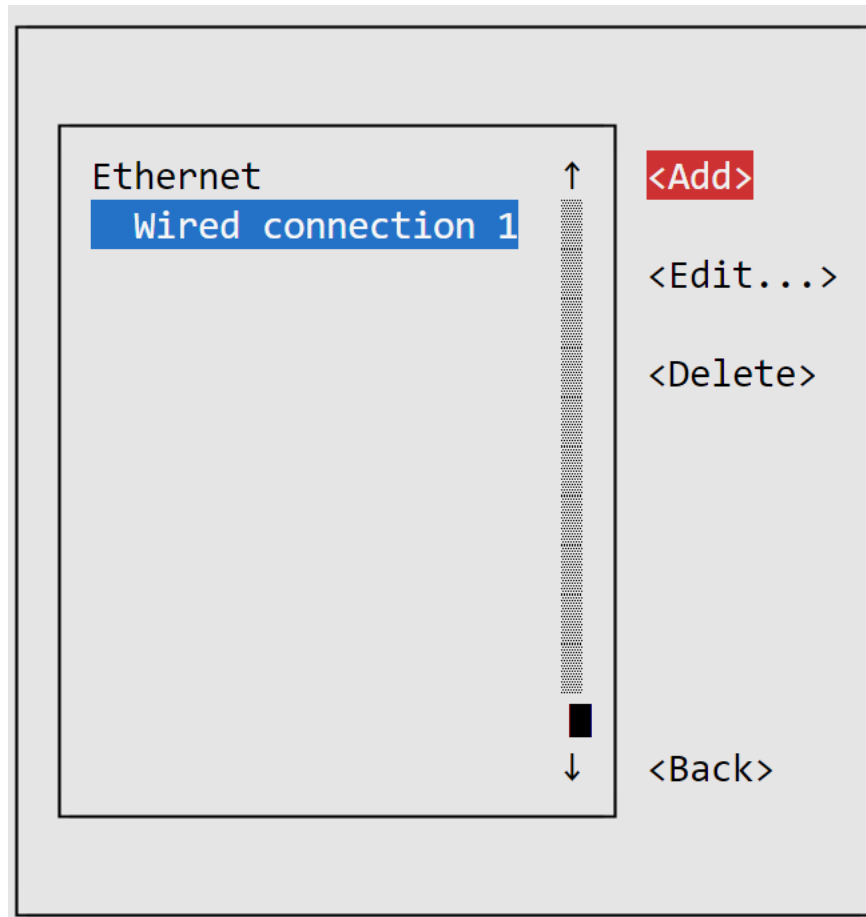
```
ecai@raspberrypi:~/workdir $ sudo nmtui
```

From the terminal, run:
sudo nmtui

Local Env. Setup: PoliTO WiFi



Local Env. Setup: PoliTO WiFi



Local Env. Setup: PoliTO WiFi

Edit Connection

Profile name

Device

WI-FI <Hide>

SSID

Mode

Security

Authentication

Anonymous identity

Domain

CA cert

CA cert password

☐ Show password

PEAP version

Local Env. Setup: PoliTO WiFi

Edit Connection

Anonymous identity sXXXXXX@studenti.polito.it

Domain

CA cert

CA cert password

☐ Show password

PEAP version <Automatic>

Inner authentication <MSCHAPv2>

Username sXXXXXX@studenti.polito.it

Password *****

☐ Show password

<Store password for all users>

BSSID

Cloned MAC address

MTU (default)

- 6. Enter the *Username*
- 7. Enter the *Password*

Local Env. Setup: PoliTO WiFi

Edit Connection

☐ Show password
<Store password for all users>

BSSID

Cloned MAC address

MTU (default)

= IPv4 CONFIGURATION <Automatic> <Show>

= IPv6 CONFIGURATION <Automatic> <Show>

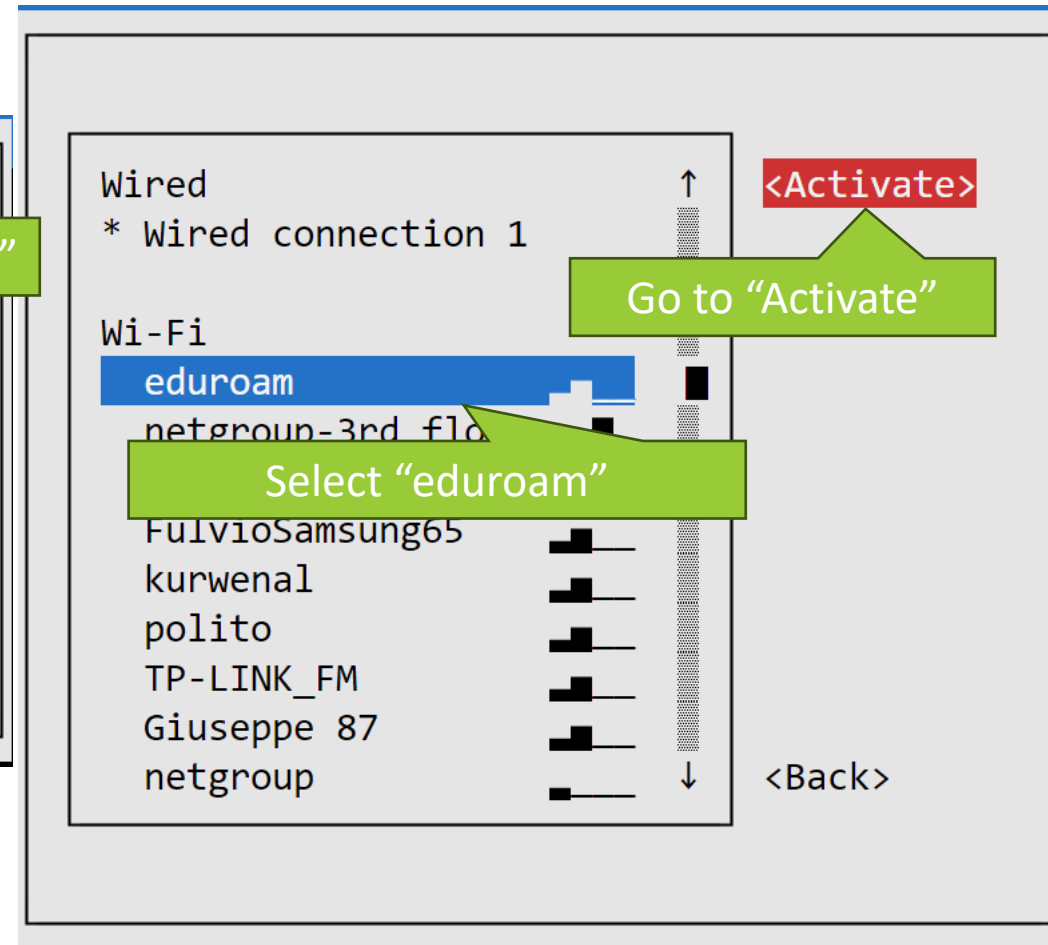
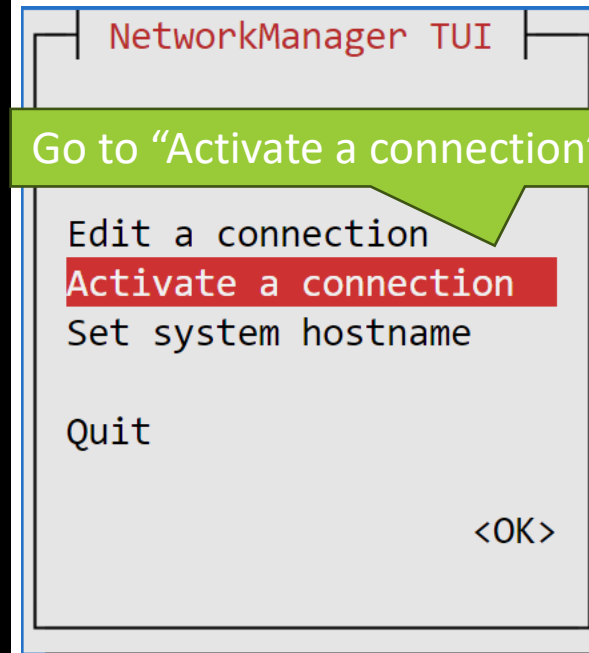
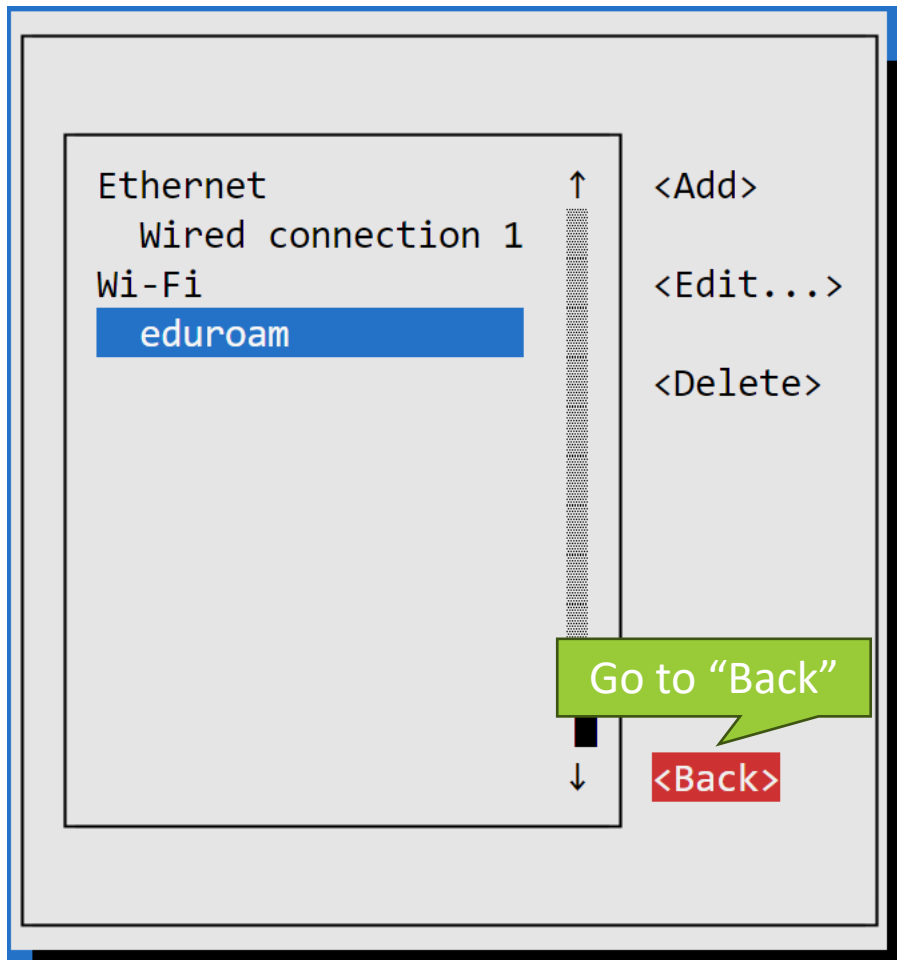
☒ Automatically connect

☒ Available to all users

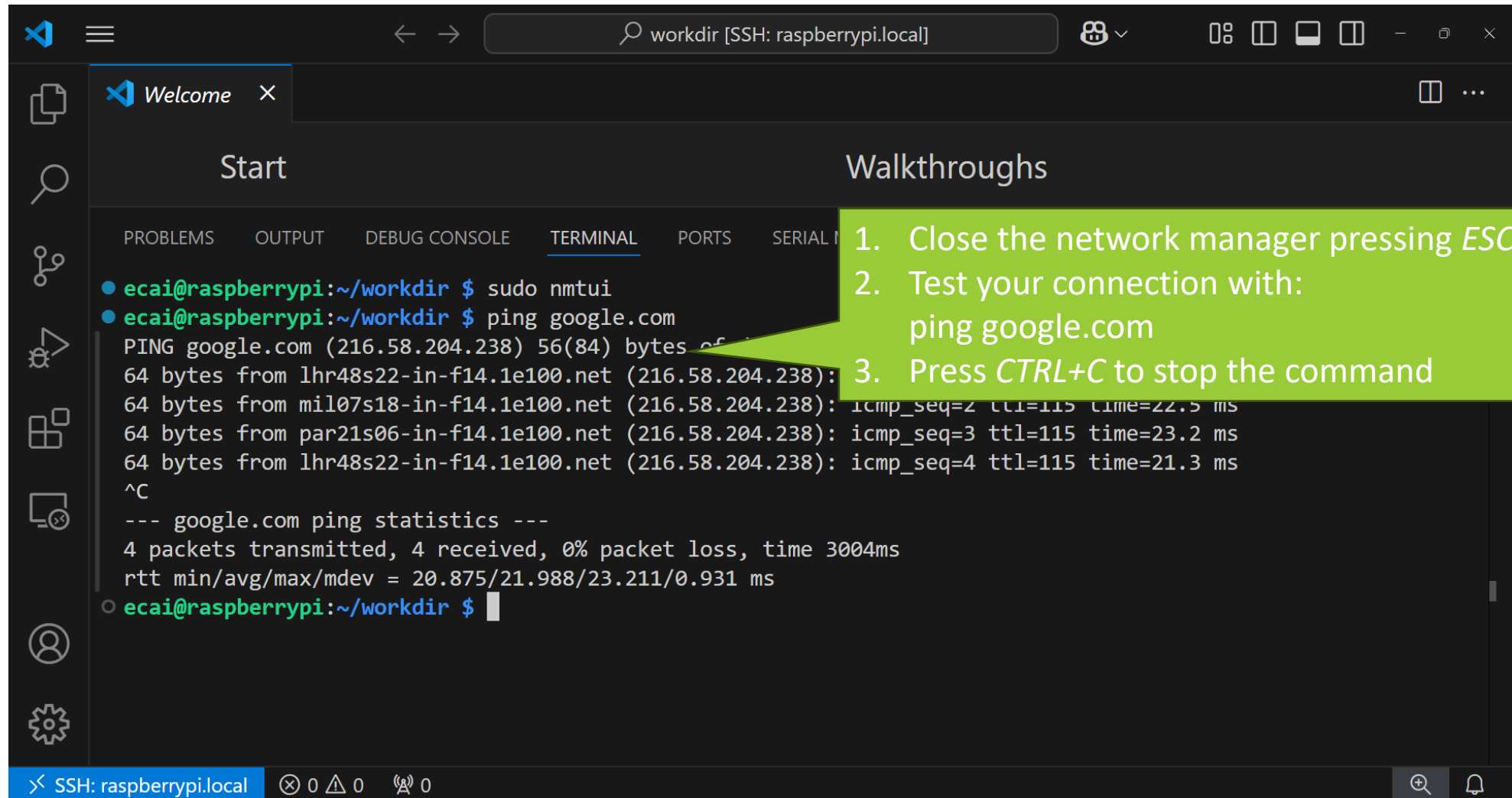
<Cancel> **<OK>**

8. Click "OK"

Local Env. Setup: PoliTO WiFi



Local Env. Setup: PoliTO WiFi



The screenshot shows a Visual Studio Code window with a terminal connected to a Raspberry Pi via SSH. The terminal displays the execution of `sudo nmtui` and `ping google.com`. The ping command shows four successful packets with 0% loss. A green callout box with three numbered steps is overlaid on the terminal output.

```
workdir [SSH: raspberrypi.local]

Welcome x

Start Walkthroughs

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL I
• ecai@raspberrypi:~/workdir $ sudo nmtui
• ecai@raspberrypi:~/workdir $ ping google.com
PING google.com (216.58.204.238) 56(84) bytes of data:
64 bytes from lhr48s22-in-f14.1e100.net (216.58.204.238): icmp_seq=1 ttl=115 time=22.5 ms
64 bytes from mil07s18-in-f14.1e100.net (216.58.204.238): icmp_seq=2 ttl=115 time=22.5 ms
64 bytes from par21s06-in-f14.1e100.net (216.58.204.238): icmp_seq=3 ttl=115 time=23.2 ms
64 bytes from lhr48s22-in-f14.1e100.net (216.58.204.238): icmp_seq=4 ttl=115 time=21.3 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 20.875/21.988/23.211/0.931 ms
• ecai@raspberrypi:~/workdir $
```

1. Close the network manager pressing *ESC*
2. Test your connection with:
ping google.com
3. Press *CTRL+C* to stop the command

SSH: raspberrypi.local 0 0 0

Local Env. Setup: Home WiFi / Hotspot

Note: Mobile Hotspot will be needed for some exercises while working in class

1. In the terminal, run `sudo nmtui`
2. Go to *Edit a connection*
3. Go to *Add*
4. Go to *Wi-Fi*
5. Set *Profile name* to the name of your personal connection
6. Set *SSID* to the name of your personal connection
7. Set *Security* to *WPA & WPA2 Personal*
8. Enter the password in the *Password* field
9. Go to *OK*. Activate the connection and exit from the network manager

Exercises

- Download “**LAB1 – Exercises.pdf**” from the folder “**labs/lab1-getting-started**” in the Material section of the Teaching Portal
- Enjoy working on the exercises:
 1. **Monitor T/H with Python**
 2. **Record Audio with the RPI & USB Microphone**